

Exam-Focused Notes (Short & Clear)

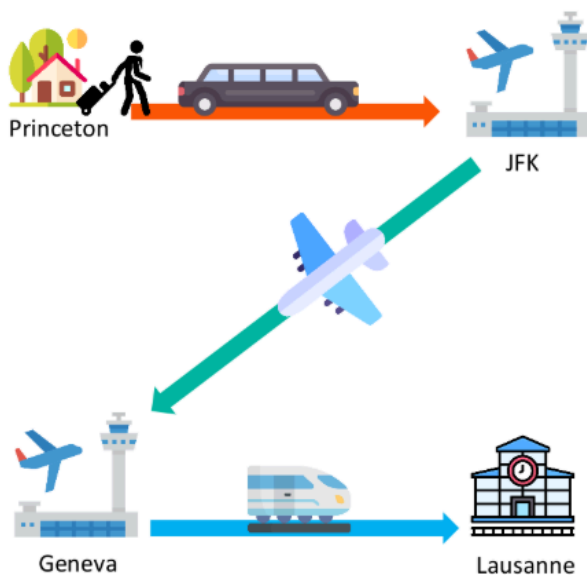
1. Link Layer Basics

- **Node:** Any device running a link-layer protocol (host, router, switch, AP).
- **Link:** A communication channel connecting adjacent nodes.
- **Role:** Move a **datagram hop-by-hop** across each link in the end-to-end path.
- **Operation:** At every hop, the datagram is encapsulated into a **link-layer frame**.

2. Transportation Analogy

- Tourist → **Datagram**
- Travel segments → **Links**
- Transport modes (car/plane/train) → **Link-layer protocols**
- Travel agent → **Routing protocol**
- Key idea: Each link may use a different technology, but all move data between adjacent nodes.

Transportation analogy



transportation analogy:

- trip from Princeton to Lausanne
 - limo: Princeton to JFK
 - plane: JFK to Geneva
 - train: Geneva to Lausanne
- tourist = **datagram**
- transport segment = **communication link**
- transportation mode = **link-layer protocol**
- travel agent = **routing algorithm**

3. Link Layer Services

(a) Framing

- Encapsulates the network-layer datagram in a **frame**.
- Frame = **Header + Data**.
- Structure depends on link-layer protocol (e.g., Ethernet, WiFi).

(b) Link Access (MAC protocols)

- Controls how nodes share the link.
- For **point-to-point links** → send anytime.
- For **broadcast links** → MAC used to avoid collisions (multiple access problem).

(c) Reliable Delivery

- Uses ACKs and retransmissions.
- Useful in **wireless links** (high error rate).
- Often **not used** in wired links (fiber/copper) because error rate is low.

(d) Error Detection and Correction

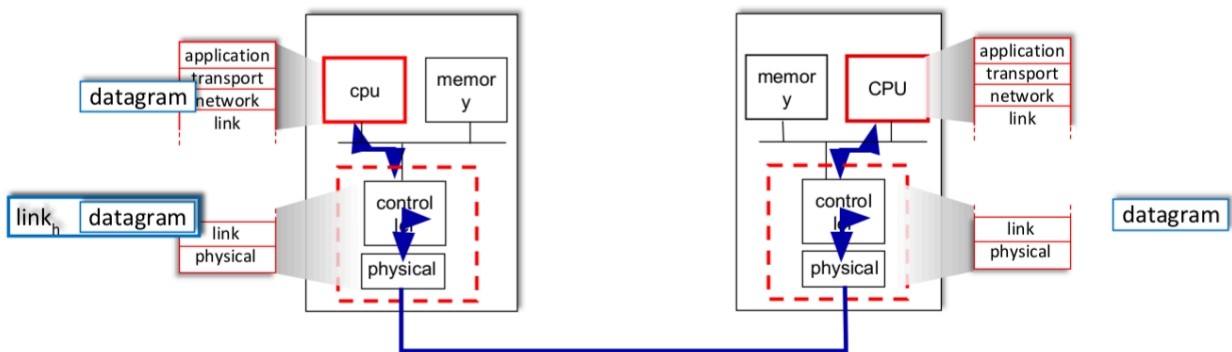
- Receivers may detect/correct bit errors caused by noise/attenuation.
- Transmitter adds **error detection bits** (e.g., CRC).
- Receiver checks for errors.
- **Error correction**: receiver identifies and fixes bit errors.

6.1.2 Where Is the Link Layer Implemented?

- **Implementation Location:**
 - The link layer is **mostly implemented in hardware**, on a chip called a **Network Interface Controller (NIC)** or network adapter.
 - Ethernet can be integrated into the motherboard or as a separate low-cost Ethernet chip.
- **Hardware Functions:**
 - Framing, link access, error detection, and other low-level link-layer functions.
 - Examples:
 - **Intel 700 series** → Ethernet protocols
 - **Atheros AR5006** → 802.11 WiFi protocols
- **Sending Side:**
 - Controller encapsulates network-layer datagrams into **link-layer frames**.
 - Fills frame fields and transmits frames according to **link-access protocol**.
 - Sets error-detection bits if error detection is used.

- **Receiving Side:**
 - Controller extracts datagram from received frame.
 - Performs error detection.
- **Software Functions:**
 - Runs on host CPU to support higher-level link-layer tasks:
 - Assembling link-layer addressing info
 - Activating controller hardware
 - Handling interrupts and errors
 - Passing datagrams up to network layer
- **Summary:**
 - Link layer = **hardware + software**
 - Acts as the **interface between hardware and software** in the protocol stack.

Interfaces communicating



sending side:

- encapsulates datagram in frame
- adds error checking bits, reliable data transfer, flow control, etc.

receiving side:

- looks for errors, reliable data transfer, flow control, etc.
- extracts datagram, passes to upper layer at receiving side

6.2 Error-Detection and -Correction Techniques — Brief Notes

Purpose

- Detect and/or correct bit errors in frames sent over a link.

- Bit errors may occur due to noise, interference, or signal distortion.
- Receiver must decide *whether an error is detected*, not whether an error actually occurred.

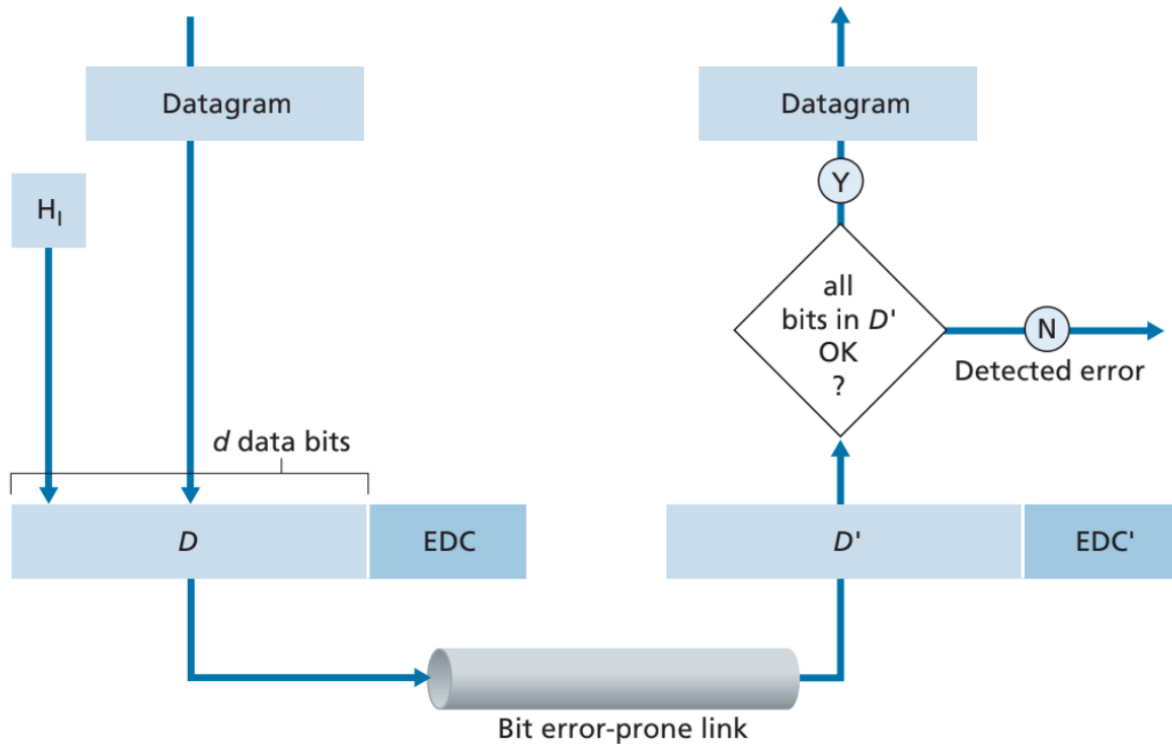


Figure 6.3 ♦ Error-detection and -correction scenario

Key Concepts

EDC (Error-Detection and -Correction) bits

- Extra bits added to data (D) before transmission.
- Help detect or correct bit errors.
- More powerful techniques → more overhead.

Types of Errors

- **Detected errors** → receiver knows corruption occurred.
- **Undetected errors** → receiver thinks data is correct but it's actually corrupted (dangerous).

Three Main Techniques

1. Parity Checks

a. Single-Bit Parity

- Add **1 parity bit** to make the number of 1s either:
 - **Even parity** → total # of 1s is even.
 - **Odd parity** → total # of 1s is odd.
- **Detection capability:** Detects **odd number** of bit errors.
- **Limitation:** Cannot detect **even number** of bit errors → may go undetected.
- Useful only when bit-error probability is very low.

2. Two-Dimensional Parity (2D parity)

- Arrange data in a **table** (i rows × j columns).
- Compute parity for:
 - Each row
 - Each column
 - Overall parity bit

Capabilities

- **Single-bit error:**
 - Detected AND corrected (Forward Error Correction – FEC).
- **Two-bit errors:**
 - Detected but NOT correctable.
- **Burst errors:**
 - More robust than 1D parity.

3. Checksums

- Used mainly in **transport layer** (e.g., TCP, UDP).
- Not covered here in detail in this part of the text.

4. Cyclic Redundancy Check (CRC)

- Stronger error-detection technique.
- Widely used in **link-layer** (Ethernet, Wi-Fi).
- Uses polynomial division on the data.

(Not detailed in the given text section but mentioned for context.)

Forward Error Correction (FEC)

- Receiver can **correct errors without retransmission**.
- Used in systems where:
 - Retransmissions are costly or slow (e.g., deep-space communication).
 - Real-time applications need minimal delay (audio/video).
- Reduces sender retransmissions and improves performance.

Summary

Technique	Detect	Correct	Notes
Single Parity	Odd # of bit errors	None	Simple, but weak
2D Parity	Single & double errors	Single-bit only	Example of FEC
Checksum	Many error patterns	None	Used at transport layer
CRC	Most strong detection	None	Used in link layer

1) Quick reminder (one line)

Parity = **one extra bit** added so the total number of **1** bits is either **even** (even parity) or **odd** (odd parity).

2) Example A — Even parity, single byte (super simple)

Data bits to send: 1 0 1 0 1 1 0

Let's count **1s digit-by-digit** (I'll show every step).

- bit1 = 1 → count = 1
- bit2 = 0 → count = 1
- bit3 = 1 → count = 2
- bit4 = 0 → count = 2

- bit5 = 1 → count = 3
- bit6 = 1 → count = 4
- bit7 = 0 → count = 4

Total number of 1s = 4, which is **even**.

Even parity rule: make total 1s even. It's already even, so **parity bit = 0**.

Transmitted bits (data + parity): 1 0 1 0 1 1 0 0

Receiver side (checks parity)

Receiver receives: 1 0 1 0 1 1 0 0

Count 1s again (digit-by-digit):

- 1 → 1
- 0 → 1
- 1 → 2
- 0 → 2
- 1 → 3
- 1 → 4
- 0 → 4
- parity bit 0 → 4

Total 1s = 4 (even) → **no error detected**.

3) Example B — Single-bit error detected

Same original data, sender sends: 1 0 1 0 1 1 0 0 (parity 0).

Suppose during transmission bit 5 flips (a single-bit error): received becomes 1 0 1 0 0 1 0 0.

Count 1s:

- 1 → 1
- 0 → 1
- 1 → 2
- 0 → 2
- 0 → 2 ← (bit5 flipped from 1->0)
- 1 → 3
- 0 → 3

- parity 0 → 3

Total 1s = 3 (odd) but we expected even → **error detected**.

Parity tells us “something’s wrong” but NOT which bit. You must request retransmission or use stronger error control.

4) Example C — Two-bit error not detected (shows limitation)

Sender sends same: 1 0 1 0 1 1 0 0. Suppose bits 2 and 5 both flip during transit:

Received: 1 1 1 0 0 1 0 0.

Count 1s:

- 1 → 1
- 1 → 2 ← bit2 flipped 0->1
- 1 → 3
- 0 → 3
- 0 → 3 ← bit5 flipped 1->0
- 1 → 4
- 0 → 4
- parity 0 → 4

Total 1s = 4 (even) → parity thinks “okay” → **error NOT detected**.

That’s why single-bit parity fails for even-numbered bit flips.

5) Two-Dimensional Parity (LRC) — arrange data in rows & columns

This is stronger. We add a parity bit at the end of **each row** and an extra parity byte for **each column**. With this you can often **detect and even correct** a single-bit error.

Example — small 3×4 data block (3 rows, 4 columns)

Original data (3 rows × 4 columns):

Row1: 1 0 1 1

Row2: 0 1 1 0

Row3: 1 1 0 0

Step 1 — compute row parity (even parity for each row)

- Row1: bits = 1 0 1 1 → count 1s = 3 (odd) → row parity = 1 (to make even)
Row1 transmitted: 1 0 1 1 | 1
- Row2: bits = 0 1 1 0 → count 1s = 2 (even) → row parity = 0
Row2 transmitted: 0 1 1 0 | 0
- Row3: bits = 1 1 0 0 → count 1s = 2 (even) → row parity = 0
Row3 transmitted: 1 1 0 0 | 0

Now form the table including row parities:

C1	C2	C3	C4	RowParity
Row1: 1	0	1	1	1
Row2: 0	1	1	0	0
Row3: 1	1	0	0	0

Step 2 — compute column parity (even parity down each column)

Count 1s column-wise:

- Col1: Row1=1, Row2=0, Row3=1 → total = 2 → column parity = 0
- Col2: 0,1,1 → total = 2 → column parity = 0
- Col3: 1,1,0 → total = 2 → column parity = 0
- Col4: 1,0,0 → total = 1 → column parity = 1

So Column parity row (Cparities): 0 0 0 1

We can also add a final parity bit for the parity-of-parities (optional). Full transmitted block (including column parities) looks like:

C1	C2	C3	C4	RowP
Row1: 1	0	1	1	1
Row2: 0	1	1	0	0
Row3: 1	1	0	0	0
ColP: 0	0	0	1	

Receiver gets a corrupted block — single-bit error occurs

Suppose bit at Row2, Col3 flips from 1 → 0. Received table:

C1	C2	C3	C4	RowP
Row1: 1	0	1	1	1
Row2: 0	1	0	0	0
Row3: 1	1	0	0	0
ColP: 0	0	0	1	

← C3 changed from 1→0

Step 1 — check row parities

- Row1: count ones = 3 + rowParity1 = 1 → total 4 (even) → OK
- Row2: count ones = (0+1+0+0) = 1; plus rowParity 0 → total = 1 → **odd** → **row parity mismatch**
- Row3: OK

Row parity mismatch at **Row2**.

Step 2 — check column parities

Compute column counts with received bits:

- Col1: 1+0+1 = 2 → plus ColP 0 → OK
- Col2: 0+1+1 = 2 → plus ColP 0 → OK
- Col3: 1+0+0 = 1 → plus ColP 0 → **mismatch**
- Col4: 1+0+0 = 1 → plus ColP 1 → OK

Column parity mismatch at **Col3**.

Intersection gives error location: Row2, Col3.

Since only one intersection, we can **flip that bit back** (0 → 1) and correct the error! 🎉

So 2-D parity not only *detects* but can *locate & correct* a **single-bit error**.

8-bit Checksum — Very Simple Solved Example

Suppose we want to send **two 8-bit words**:

Word 1 = 0101 1100

Word 2 = 0011 0011

We will compute the checksum for these.

✓ Step 1 — Add the two 8-bit words (1's complement addition)

Write them one under the other:

```
0101 1100
+ 0011 0011
-----
```

➤ Add bit-by-bit or nibble-by-nibble

Let's add the **right half** (**1100 + 0011**) first:

```
1100 (12)
+0011 (3)
-----
1111 (15)
```

Result right side = **1111** (no carry)

Now add the **left half** (**0101 + 0011**):

```
0101 (5)
+0011 (3)
-----
1000 (8)
```

Again, result = **1000**, no carry.

So the full 8-bit sum =

1000 1111

No carry-out → nothing to wrap around.

✓ Step 2 — Take 1's Complement (flip all bits)

```
1000 1111 ← sum
0111 0000 ← 1's complement (this is the checksum)
```

Checksum = 0111 0000

This is the bit value we send along with the data.

✓ Step 3 — Receiver Verification

Receiver adds:

```
Word1: 0101 1100
Word2: 0011 0011
```

Checksum: 0111 0000

=====

Sum = 1111 1111 ← all 1s means NO ERROR

Since the result is **1111 1111**, receiver accepts the message.

Final Answer (Simple 8-bit Checksum)

Item	Value
Word 1	0101 1100
Word 2	0011 0011
Checksum	0111 0000

Cyclic Redundancy Check (CRC) — Fully Solved Example

Given

$$D = 101110, \quad G = 1001, \quad \text{length}(G) = 4 \Rightarrow r = 3$$

The sender must compute a 3-bit remainder R such that:

$$\text{TransmittedFrame} = D \parallel R$$

Step 1: Append r zeros to D

$$D \cdot 2^3 = 101110000$$

This 9-bit number is divided by generator $G = 1001$ using modulo-2 XOR division.

Step 2: Sender Side Modulo-2 Division

Modulo-2 division means:

$$1 \oplus 1 = 0, \quad 1 \oplus 0 = 1, \quad 0 \oplus 0 = 0$$

(no carries, no borrows)

$$\underline{1001) 101110000}$$

1) Divide first 4 bits: $1011 \oplus 1001 = 0010$

Bringdownnextbit :00101

2) Since first bit is 0, shift window: 0100

Bringdownnextbit :01010

3) Still cannot divide (starts with 0). Shift again: 1000

Nowdivide :1000 $\oplus$ 1001 = 0011

Bringdownnextbit :110

4) Shift until first 1 appears: 1100

Divide :0011 $\oplus$ 1100 = 1001(*cannot, so shift*)

Correctnextwindow :1010

$$1010 \oplus 1001 = 0011$$

The last $r = 3$ bits are the remainder:

$$R = 011$$

Thus the sender transmits:

$$Framesent = D \parallel R = 101110011$$

Step 3: Receiver Side Checking

Receiver receives:

$$101110011$$

Receiver divides it by the same generator $G = 1001$.

Because the sender chose R so that the transmitted frame is perfectly divisible:

$$101110011 \div 1001 \Rightarrow Remainder = 000$$

Step 4: Error Decision

$$If remainder = 000 \Rightarrow Noerror(ACCEPT)$$

$$If remainder \neq 000 \Rightarrow Error detected(REJECT)$$

Since remainder is:

$$000 \Rightarrow \text{Data Accepted}$$

Final Conclusion

- Sender divides 101110000 by 1001
- Remainder found: $R = 011$
- Transmitted frame: 101110011
- Receiver re-divides and gets remainder 000
- No error is detected

CRCCheckPassedSuccessfully

6.3 Multiple Access Links and Protocols — Easy Notes

1. Types of Links

There are **two types of network links**:

1. Point-to-Point Link

- Only **one sender** and **one receiver**.
- Example:
 - PPP
 - HDLC
 - Fiber connection between two routers

2. Broadcast Link

- **Many** devices share **one common channel**.
- When 1 node sends, **everyone hears** it.
- Examples:
 - Ethernet LAN (old shared coax cable)
 - WiFi
 - Satellite link

Why Do We Need Multiple Access Protocols?

Because on broadcast channels:

More than one node may transmit at the same time

Their transmitted signals crash into each other

This is called a **collision**

When collision happens:

- Frames get **garbled**
- **Nobody can understand anything**
- Channel time is **wasted**

So we need protocols that **control who sends, when**.

Human Analogy (VERY EASY)

Think of a **classroom or a party**:

- If everyone speaks at the same time → **noise / collision**
- So humans use rules:
 - “Raise your hand”
 - “Don’t interrupt”
 - “Wait for your turn”
 - “Don’t speak continuously”

Computers need **similar rules** using **multiple access protocols**.

Where are Multiple Access Channels Used?

- **WiFi networks**
- **Shared coaxial cable systems**
- **Satellite communication**
- **Old Ethernet LANs**

Basically anywhere multiple nodes share **one channel**.

What happens during a collision?

If two nodes send:

- All nodes receive **mixed-up signals**
- None of the frames can be understood
- The channel becomes **useless** during collision time

- Data must be **retransmitted**

So we need protocols to **avoid** or **handle** collisions.

Categories of Multiple Access Protocols

Almost all multiple access protocols fall into **three main categories**:

1 Channel Partitioning Protocols

- Divide the channel into equal pieces:
 - Time slots
 - Frequency bands
 - Code sequences
- Everyone gets a piece → **no collisions**
- BUT:
 - If only 1 device is active, its speed is low

2 Random Access Protocols

- Nodes transmit whenever they want
- Collisions are **possible**
- Examples:
 - ALOHA
 - CSMA/CD
 - CSMA/CA (WiFi)

Benefit:

- High speed when only few nodes active

3 Taking-Turns Protocols

- Nodes **take turns** sending data
- Controlled but fair
- Examples:
 - Polling
 - Token passing

Good when many nodes always have data.

Ideal Properties of a Multiple Access Protocol

A good protocol should:

✓ **1. Give full speed (R bps) to one sender**

If only one node is transmitting, it should get **entire bandwidth**.

✓ **2. Share equally when many nodes transmit**

If **M nodes** want to send, each should get **R/M on average**.

✓ **3. Be decentralized**

- No master controller
- No single point of failure

✓ **4. Be simple**

- Easy to implement
- Low cost

6.3 Multiple Access Links and Protocols – EASY NOTES

Nodes = computers sharing **one common link**

Problem = if 2 nodes send at same time → **collision** → data lost.

So we need rules called **multiple access protocols**.

6.3.1 Channel Partitioning Protocols (EASY)

These protocols **divide the channel into equal parts** so that nodes **never collide**.

Three types:

- ① **TDM – Time Division Multiplexing**
- ② **FDM – Frequency Division Multiplexing**
- ③ **CDMA – Code Division Multiple Access**

Let's explain them one by one.

① **TDM (Time Division Multiplexing)**

Easy idea:

Each node gets a **turn** to speak at a specific time.

Like a teacher giving.

- 1st second → student A
- 2nd second → student B
- 3rd second → student C
- Then repeat

How it works:

- Time is divided into **frames**
- Each frame has **N time slots**
- Slot i belongs to node i
- A node can send **only in its slot**

Example:

Channel rate = **R bps**

Number of nodes = **N = 4**

Then each node gets $R/4$ bps.

Advantages:

- ✓ **No collision**
- ✓ **Fair** (everyone gets equal time)

Disadvantages:

- ✗ If a node has lots of data, it must **wait** for its turn
- ✗ If only 1 node wants to send, it still gets only **R/N** rate → waste
- ✗ Slow when traffic is uneven

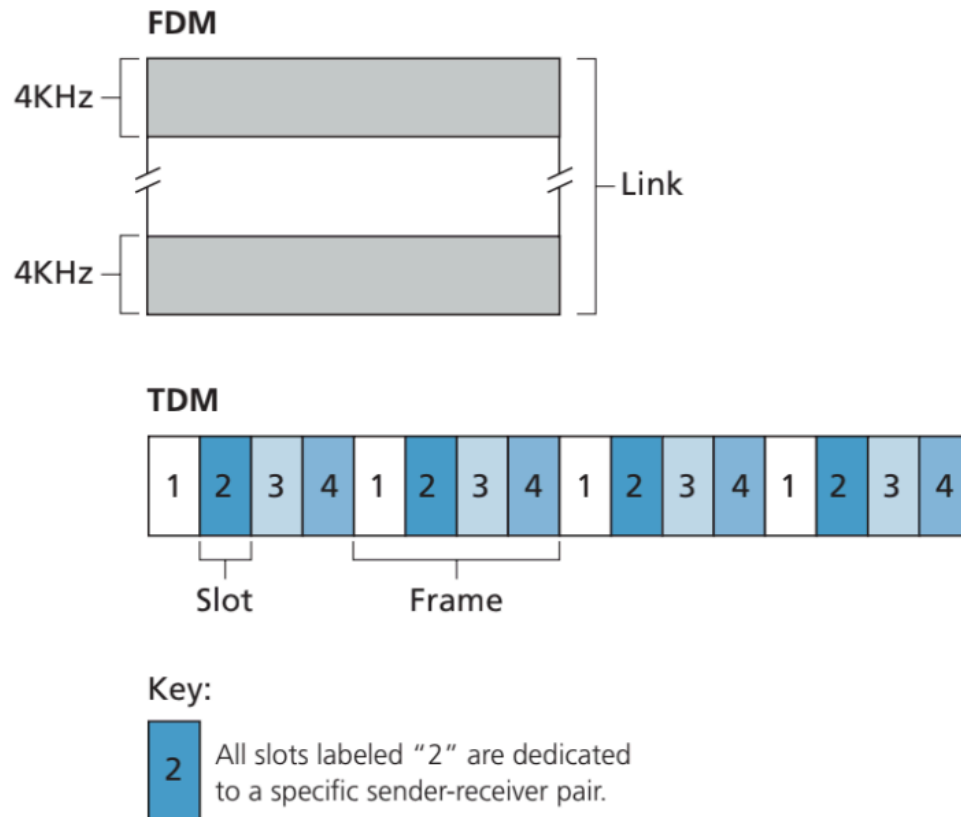


Figure 6.9 ♦ A four-node TDM and FDM example

2 FDM (Frequency Division Multiplexing)

💡 **Easy idea:** All nodes talk **simultaneously**, but on **different frequencies**.

Example: Radio stations: FM 98, FM 101, FM 103 — all broadcast at same time, but on different frequencies.

How it works:

- Divide channel into **N frequency bands**
- Each band has **R/N bps**
- Assign each frequency to one node

👍 **Advantages:**

- ✓ No collisions
- ✓ Fair division

Disadvantages:

- ✗ Still limited to **R/N**
- ✗ Wasteful if some users are silent
- ✗ Need hardware for frequency separation

3 CDMA (Code Division Multiple Access)

Easy idea:

Everyone speaks at the same time, but using **different secret codes**.

Imagine:

- Node A speaks English
 - Node B speaks Urdu
 - Node C speaks Chinese
- Even if all speak simultaneously, a listener of English only hears English.

How it works (simple):

- Each node gets a **unique code**
- Receiver knows the sender's code
- Can separate mixed signals mathematically

Advantages:

- ✓ Multiple nodes can send at the same time
- ✓ No collisions
- ✓ Good security (used in military radios)
- ✓ Used in mobile networks

Disadvantages:

- ✗ More complex
- ✗ Codes must be selected carefully

Summary of Channel Partitioning

Technique	How?	Collision ?	Speed when alone?	Fair?
TDM	Divide time	✗ No	✗ Slow (R/N)	✓ Yes
FDM	Divide frequency	✗ No	✗ Slow (R/N)	✓ Yes
CDMA	Divide codes	✗ No	✓ Can use full R	✓ Yes

6.3.2 Random Access Protocols (SUPER EASY)

In these protocols:

- ✓ **Nodes transmit whenever they have data**
- ✓ **Collisions can happen**
- ✓ **If collision happens → retransmit after a random delay**

Why random?

If two nodes retry immediately → they collide again.

Random delay reduces chance of repeated collisions.

Examples:

- ALOHA
- Slotted ALOHA
- CSMA
- Ethernet (CSMA/CD)
- WiFi (CSMA/CA)

Random access protocols

- when node has packet to send
 - transmit at full channel data rate R
 - no *a priori* coordination among nodes
- two or more transmitting nodes:
“collision”
- **random access protocol** specifies:
 - how to detect collisions
 - how to recover from collisions (e.g., via delayed retransmissions)
- examples of random access MAC protocols:
 - ALOHA, slotted ALOHA
 - CSMA, CSMA/CD, CSMA/CA

Slotted ALOHA (Beginner Explanation)

Rules:

- Time is divided into **slots**
- Each slot = time to send **1 frame**
- Node can transmit **only at start of a slot**

If:

- Only one node transmits → **success**
- Two or more transmit → **collision** → entire slot wasted
- After collision → each node tries again with probability p

Analogy:

Imagine:

- Everyone can speak only at exact seconds (1.0 sec, 2.0 sec, 3.0 sec...)
- If two speak at the same second → noise → try again randomly

Efficiency of Slotted ALOHA

When many nodes are active:

- About **37%** of slots are successful
- About **37%** are empty.
- About **26%** have collisions

So maximum efficiency = $1/e \approx 0.37$

Meaning:

If channel is 100 Mbps → useful throughput = **37 Mbps**

FINAL EASY SUMMARY

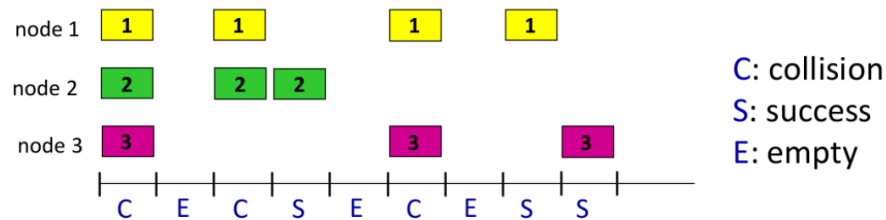
Channel Partitioning Protocols:

- Divide channel (time / frequency / code)
- No collisions
- Mostly wasteful when few nodes active
- Slow when only one sender wants to transmit

Random Access Protocols:

- Let nodes transmit anytime
- Collisions happen but resolved using randomness
- Very efficient when few nodes active
- But not perfect when many nodes are active

Slotted ALOHA



Pros:

- single active node can continuously transmit at full rate of channel
- highly decentralized: only slots in nodes need to be in sync
- simple

Cons:

- collisions, wasting slots
- idle slots
- nodes may be able to detect collision in less than time to transmit packet
- clock synchronization

Slotted ALOHA: efficiency

efficiency: long-run fraction of successful slots (many nodes, all with many frames to send)

- *suppose:* N nodes with many frames to send, each transmits in slot with probability p
 - prob that given node has success in a slot $= p(1-p)^{N-1}$
 - prob that *any* node has a success $= Np(1-p)^{N-1}$
 - max efficiency: find p^* that maximizes $Np(1-p)^{N-1}$
 - for many nodes, take limit of $Np^*(1-p^*)^{N-1}$ as N goes to infinity, gives:

max efficiency $= 1/e = .37$

- **at best:** channel used for useful transmissions 37% of time!



Link Layer 27

FIRST: What are we trying to calculate?

In **Slotted ALOHA**:

- Many nodes share the same channel
- A node chooses to transmit in a slot with probability p

- Collisions occur if ≥ 2 nodes transmit
- A slot is **successful** only if **exactly ONE** node transmits

We want to know:

➤ **What fraction of slots, in the long run, result in EXACTLY ONE success?**

This is called **efficiency**.

Step 1: Probability a specific node succeeds

For **Node A** to succeed in a slot:

1. Node A must **transmit** $\rightarrow$ probability = p
2. ALL other $(N-1)$ nodes must **NOT transmit** $\rightarrow$ probability = $(1 - p)^{(N-1)}$

So:

$$P(\text{Node A succeeds}) = p(1-p)^{N-1}$$

Example:

If Node A transmits ($p = 0.1$), and 2 other nodes do NOT transmit:

$$0.1 \times (0.9)^2$$

Step 2: Probability ANY of the N nodes succeed

There are **N nodes**, and each has probability:

$$p(1-p)^{N-1}$$

So:

$$P(\text{ANY node succeeds}) = Np(1-p)^{N-1}$$

This is the **efficiency formula**.

Step 3: What value of p gives the maximum efficiency?

We need to choose p so that:

$$Np(1-p)^{N-1}$$

becomes **as large as possible**.

The mathematically best value is:

$$P = 1/N$$

This means:

➤ **When many nodes share the channel, each should transmit very rarely.**

If there are 100 nodes, each node should transmit with probability **1/100** each slot.

Step 4: Now plug in $p = 1/N$

Efficiency formula:

$$Np(1-p)^{N-1}$$

Set:

$$p=1/N$$

Then:

$$N(1/N)(1-1/N)$$

Simplify:

$$=1 \cdot (1-1/N)^{N-1}$$

So MAX efficiency =

$$(1-1/N)^{N-1}$$

Step 5: Take the limit as $N \rightarrow \infty$

The expression:

$$(1-1/N)^{N-1}$$

is a famous limit from mathematics:

$$(1-1/N)^N \approx 1/e$$

And:

$$1/e \approx 0.3679 \approx 0.37$$

FINAL RESULT

Maximum efficiency of Slotted ALOHA = $1/e \approx 0.37$

BUT WHAT DOES THIS MEAN IN SIMPLE ENGLISH?

Even when Slotted ALOHA is used perfectly:

✓ **Only 37% of the time is the channel doing useful work**

✗ **63% of the time is wasted (empty slots or collisions)**

Why?

Because:

- If too many nodes transmit → collisions
- If too few nodes transmit → empty slots

The value $p = 1/N$ gives the balance:

- Not too aggressive
- Not too passive

But even then → **waste is unavoidable**, because nodes are independent.

SUPER SIMPLE ANALOGY (you will love this)

Imagine 100 students taking turns to speak.

Each second, every student decides:

- “Should I speak this second?” with probability **1/100**

What happens?

- Sometimes **nobody** speaks → empty slot
- Sometimes **two speak** → collision
- Sometimes **exactly one speaks** → success

With the best possible choice (1/100),

→ **Only 37% of the seconds have exactly ONE speaker.**

This is Slotted ALOHA.

What is Pure ALOHA?

- The **oldest** multiple-access protocol (1970).
- Fully **decentralized** → nodes do NOT synchronize with anyone.
- A node **sends whenever it wants**, without checking the channel.

What happens if two nodes transmit at overlapping times?

→ **Collision**

→ Both frames become **garbage**

→ Must retransmit later (random time)

When does a frame succeed?

A frame starts at time t_0 .

To be successful:

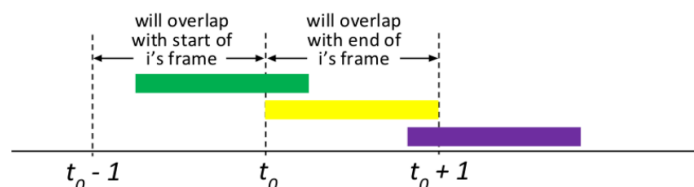
- No other node should begin transmitting in the time interval $[t_0 - 1, t_0]$ → would overlap with the start
- No other node should begin transmitting in the interval $[t_0, t_0 + 1]$ → would overlap with the end

Total “danger window” = **2 frame times**

If ANY node starts transmitting in these 2 slots → collision.

Pure ALOHA

- unslotted Aloha: simpler, no synchronization
 - when frame first arrives: transmit immediately
- collision probability increases with no synchronization:
 - frame sent at t_0 collides with other frames sent in $[t_0 - 1, t_0 + 1]$



- pure Aloha efficiency: 18% !

Efficiency of Pure ALOHA

Only **18%** ($1/(2e)$) of slots are useful.

Why so low?

- Nodes transmit at random
- Huge collision window ($2 \times$ frame time)

SLOTTED ALOHA — SIMPLE NOTES

What is Slotted ALOHA?

- All transmissions **MUST** start at the **beginning of a time slot**.
- Nodes are synchronized.
- Collisions can only happen if **two nodes pick the same slot**.

Cartoon understanding

Slots: |---1---|---2---|---3---|---4---|

Node can only start sending at beginning of a slot.

Efficiency of Slotted ALOHA

About **37%** ($1/e$).

Why is it double Pure ALOHA?

- Collision window is only **1 frame time**, not 2.

Slotted ALOHA = more organized → fewer collisions.

CSMA — SIMPLE NOTES (Carrier Sense Multiple Access)

CSMA = **Listen before talking**

Two main rules (just like humans!)

1. Carrier Sensing

- Node listens to the channel.
- If someone is talking (channel busy), node waits.

- If quiet, node starts transmitting.

2. Collision Detection (CSMA/CD)

- Node keeps listening WHILE transmitting.
- If it senses another node → **collision detected**.
- Immediately stops sending → reduces wasted time.
- Reschedules using random backoff.

Human example:

In class:

- You **listen** before asking a question.
- If you start speaking and someone else **ALSO** starts → both stop, then try again.

This is CSMA/CD → used in **Ethernet**.

DIFFERENCE BETWEEN ALOHA AND CSMA

Protocol	Do nodes listen before transmitting?	Do they detect collision while transmitting?	Efficiency
Pure ALOHA	✗ No	✗ No	~18%
Slotted ALOHA	✗ No (but synchronized slots)	✗ No	~37%
CSMA	✓ Yes	✗ No	Higher
CSMA/CD	✓ Yes	✓ Yes	Highest (close to 100% when low load)

ETHERNET CSMA/CD — SUPER EASY NOTES

Ethernet uses **CSMA/CD**, which means:

- **CS** = Carrier Sense → Listen before talking
- **MA** = Multiple Access → Many nodes share the same channel
- **CD** = Collision Detection → If two talk at the same time → STOP

Think of it like polite people talking in a room.

HOW CSMA/CD WORKS (Step-by-Step)

Step 1 — Ethernet gets a frame from the Network Layer

Network layer → gives datagram

Ethernet → places it in a frame.

Step 2 — Listen to the channel (Carrier Sense)

- If **idle** → start sending immediately.
- If **busy** → wait until it becomes idle → then start transmitting.

Already better than ALOHA, because ALOHA does NOT listen before sending.

Step 3 — While transmitting, Ethernet keeps listening

This is the **CD (Collision Detection)** part.

If Ethernet hears no interference →

 **Success! Frame delivered.**

If Ethernet hears another signal →

 **Collision happened.**

It immediately does two things:

Step 4 — Abort transmission + Send Jam Signal

- Abort = stop transmitting immediately
- Jam signal = "Collision happened! Everyone stop!!"

This ensures all nodes know a collision occurred.

Step 5 — Binary Exponential Backoff

After a collision, node must wait a **random time** before trying again.

This prevents the same nodes from colliding repeatedly.

Rules:

After **m-th collision**, choose:

$K \in \{0, 1, 2, \dots, 2^m - 1\}$

Then wait for:

Wait time = $K \times 512$ bit times

Why 512 bit times?

Because it is the minimum time needed to ensure the collision is detectable.

KEY IDEA OF BACKOFF

- After 1 collision → choose K from {0,1}
- After 2 collisions → choose K from {0,1,2,3}
- After 3 collisions → choose K from {0 to 7}
- After m collisions → range becomes bigger → more spacing between retries

More collisions → bigger waiting window → less chance to collide again

Why is CSMA/CD more efficient than ALOHA?

Because:

Feature	ALOHA	CSMA/CD
Do nodes listen before sending?	✗ No	✓ Yes
Stop sending when collision happens?	✗ No	✓ Yes
Collision window	Large	Very small
Efficiency	18–37%	Up to 100% in ideal conditions

Understanding CSMA/CD Efficiency

Efficiency depends on two things:

- **t_{prop}** = propagation delay (time signal takes to reach farthest node)
- **t_{trans}** = transmission time (time needed to send entire frame)

Efficiency formula (simplified idea):

$$\text{Efficiency} = 1 / (1 + 5 \cdot t_{\text{prop}} / t_{\text{trans}})$$

What this means:

1) If propagation delay $\rightarrow 0$

$$t_{\text{prop}} \rightarrow 0$$

$$\text{Efficiency} \rightarrow 1 \text{ (100\%)}$$

Why?

Nodes instantly hear each other $\rightarrow$ collisions rarely happen.

2) If frame transmission time $\rightarrow$ very large

$$t_{\text{trans}} \rightarrow \infty$$

$$\text{Efficiency} \rightarrow 1 \text{ again.}$$

Why?

Once a node starts transmitting, it occupies the channel for long $\rightarrow$ little overhead.

So CSMA/CD is efficient because:

- Nodes listen before sending $\rightarrow$ fewer collisions
- They stop immediately when collisions happen $\rightarrow$ waste is small
- They wait random backoff time $\rightarrow$ avoid repeated collisions
- Very simple hardware $\rightarrow$ cheap
- Fully decentralized $\rightarrow$ no central controller required

That's why old Ethernet (wired LANs) used CSMA/CD.

QUICK SUMMARY FOR EXAM (MEMORIZE)

CSMA/CD Algorithm:

1. Sense channel.

- If idle: transmit

- If busy: wait until idle, then send

2. While transmitting, keep sensing.

3. If collision detected:

- Abort transmission

- Send jam signal

- Use binary exponential backoff:

After m-th collision choose K from $\{0 \dots 2^m - 1\}$

Wait $K \times 512$ bit times

4. Retry from step 1

Efficiency high when:

- $t_{\text{prop}} \rightarrow 0$
- $t_{\text{trans}} \rightarrow \text{large}$

Why better than ALOHA?

- Listen before send
- Collision detection
- Random backoff

Taking-Turns Protocols — Easy Notes

Multiple access protocols need to handle *many nodes sharing the same channel*.

Random access (ALOHA, CSMA) has collisions and wasted time $\rightarrow$ low efficiency.

So another class was created:

Taking-Turns Protocols

Idea: Instead of randomly fighting for the channel, **nodes take turns**.

This avoids collisions and wasted empty slots.

1 Polling Protocol (Master–Slave System)

How it works

- One node is the **master**.
- The master **polls** nodes one-by-one in a cycle.
- When a node is polled, it can transmit up to some **maximum number of frames**.
- Then the master polls the next node.

Example

Master $\rightarrow$ Node 1 $\rightarrow$ Node 2 $\rightarrow$ Node 3 $\rightarrow$... $\rightarrow$ Node N $\rightarrow$ back to Node 1.

Advantages

- **No collisions** $\rightarrow$ very high efficiency.
- Channel is always used productively.

✗ Disadvantages

1. Polling Delay

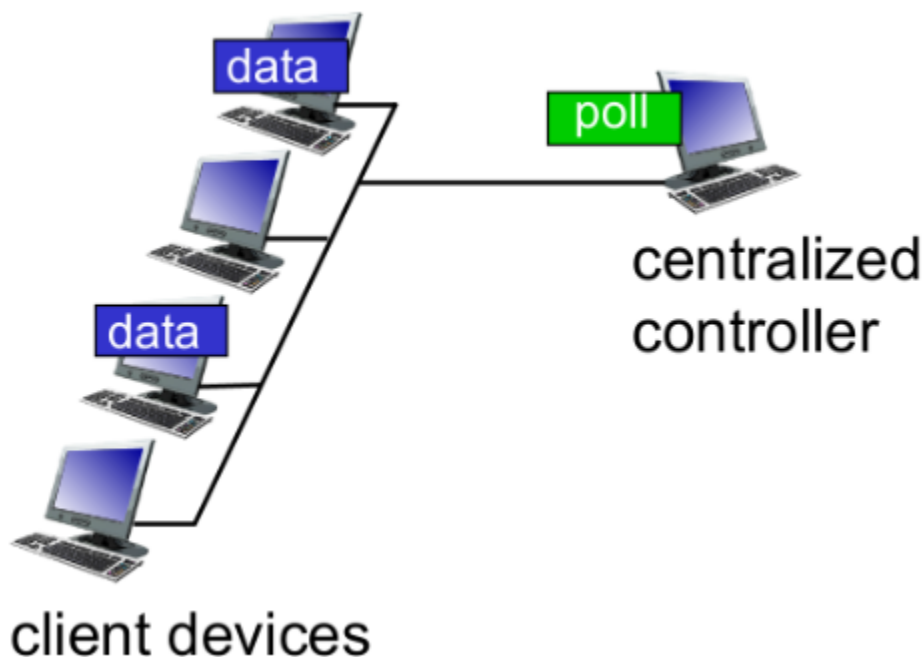
- Master must poll *every node*, even inactive ones.
- If only 1 node has data, it still waits for master to cycle through all nodes.
- So throughput is less than R bps.

2. Single Point of Failure

- If the master fails → *entire network stops working*.

Real-world example:

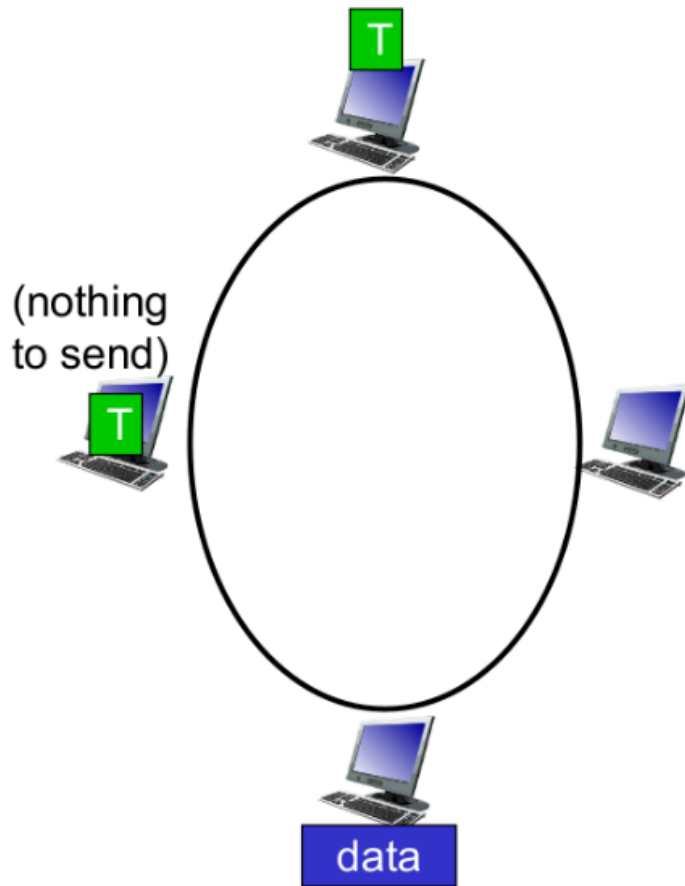
- **Bluetooth** uses a polling-based approach.



2 Token-Passing Protocol

📌 How it works

- No master node.
- A special small frame called a **token** circulates around the nodes in a fixed order.
- When a node gets the token:
 - If it has data → it transmits up to a maximum number of frames.
 - If it has no data → it immediately passes the token forward.



Advantages

- **Highly efficient**
No collisions, each node gets its turn.
- **Decentralized**
No dependency on a master node.

Disadvantages

1. **Token Loss or Failure**

- If a node fails → token may never reach the next node → network stops.
- If a node forgets to forward the token → token disappears → needs recovery.

2. **Token Management**

- Extra complexity is needed to detect or regenerate lost tokens.

Real-world examples:

- FDDI (Fiber Distributed Data Interface)
- IEEE 802.5 Token Ring

Summary in Easy Words

Protocol	How It Works	Pros	Cons
Polling	Master polls each node	No collisions, efficient	Master failure stops network, polling delay
Token Passing	Token moves node-to-node	No collisions, decentralized	Token loss, one node failure can break system

6.4 Switched Local Area Networks — Easy Notes

Switched LANs are the modern way to connect devices inside organizations (universities, offices, labs).

Unlike older broadcast LANs, switched LANs use **switches**, which operate at the **link layer (Layer 2)**.

Why “Switched LANs”?

Because frames are **forwarded (switched)** inside the network **based on MAC addresses**, not IP addresses.

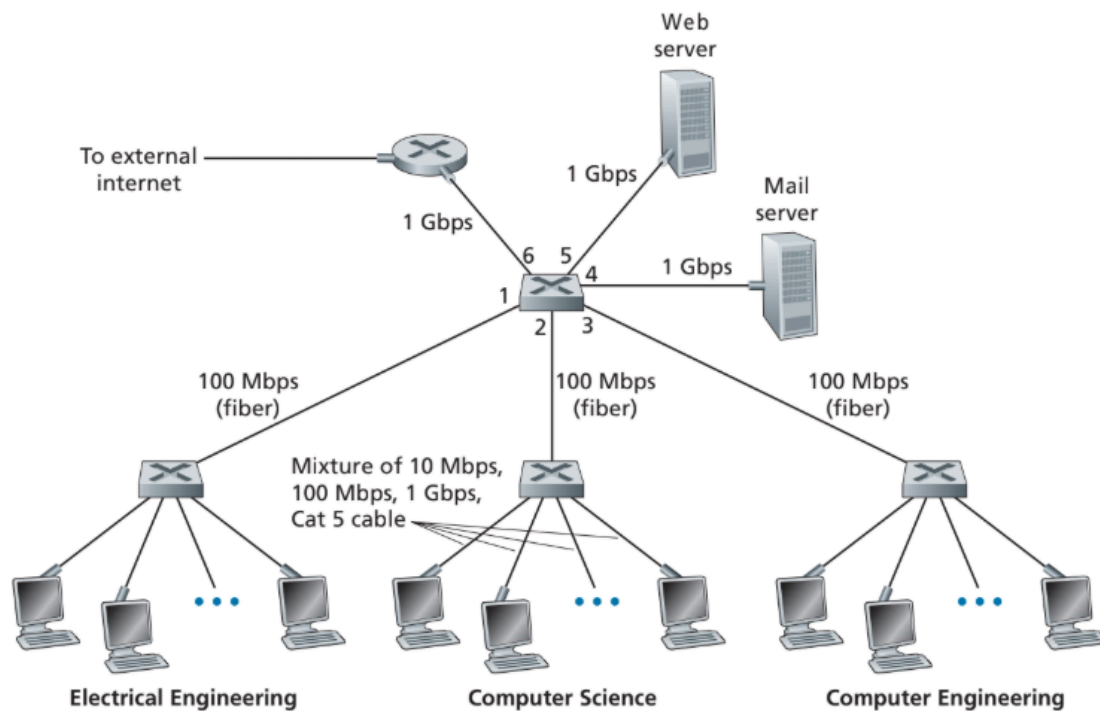


Figure 6.15 ♦ An institutional network connected together by four switches

 In Figure 6.15:

- Departments are connected to switches.
- Switches connect to servers and the router leading to the Internet.
- Switches don't run routing algorithms (OSPF, RIP).
- Switches only work with **link-layer frames**.

6.4.1 Link-Layer Addressing and ARP

Why do we need two addresses?

A device has:

- **IP address (Layer 3)** → logical, changes when you change networks.
- **MAC address (Layer 2)** → physical, stays the same everywhere.

- ✓ IP = like a **home address** (changes when you move)
- ✓ MAC = like a **national ID / CNIC number** (unique for life)

Both are needed.

MAC Addresses (Media Access Control Addresses)

Key Points:

- Assigned to **network adapters**, not hosts.
- Size: **6 bytes = 48 bits = 2^{48} possible addresses**.
- Written in **hexadecimal**, e.g.:
5C-66-AB-90-75-B1

Uniqueness

How do manufacturers avoid duplicates?

- IEEE gives each company a **block of 2^{24} MAC addresses**.
- First 24 bits = company ID (OUI)
- Last 24 bits = unique serial numbers created by the company.

Flat vs. Hierarchical Addresses

Address Type	Example	Structure	Changes When Moving?
MAC	5C-66-AB-90-75-B1	Flat	✗ No
IP	192.168.1.2	Network + Host parts	✓ Yes

So:

- MAC = permanent identity
- IP = location/address on the Internet

How MAC Addresses Are Used

When sending a frame on a LAN:

- Sender puts **destination MAC** inside the frame.
- Frame travels to all devices on the LAN, OR is forwarded by switches.
- Devices check: "Is this MAC mine?"
 - Yes → accept frame and pass datagram to higher layer.
 - No → drop.

Broadcast MAC Address

Sometimes a sender wants *every device* on the LAN to receive the frame.

Special address for this:

FF-FF-FF-FF-FF-FF (all bits = 1)

Used for:

- ARP requests
- DHCP discovery
- Network announcements

Address Resolution Protocol (ARP)

Problem:

To send something to an IP address, you need its **MAC address** first.

➡ ARP solves this by mapping:

IP → MAC

When ARP is Needed

Example:

Host A wants to send a packet to Host B:

1. A knows B's **IP address**.
2. A does **not** know B's **MAC address**.

A broadcasts an **ARP request**:

Who has IP 10.1.1.2? Tell 10.1.1.1

- 3.
4. All devices receive it.

Only the device with that IP replies:

I am 10.1.1.2, my MAC is 49-BD-D2-C7-56-2A

- 5.
6. Host A saves it in its **ARP cache** (temporary memory).

Now A can send frames directly to B.

ARP Table / ARP Cache

Stored in every host/router.

Contains pairs:

IP → MAC

Entries expire after a few minutes to avoid stale mappings.

Why ARP Is Important

Because **IP can't deliver packets directly**.

Packets must be placed inside **frames**, and frames require **MAC addresses**.



So the flow is:

Application → Transport → Network (IP address) → Link Layer (needs MAC!)



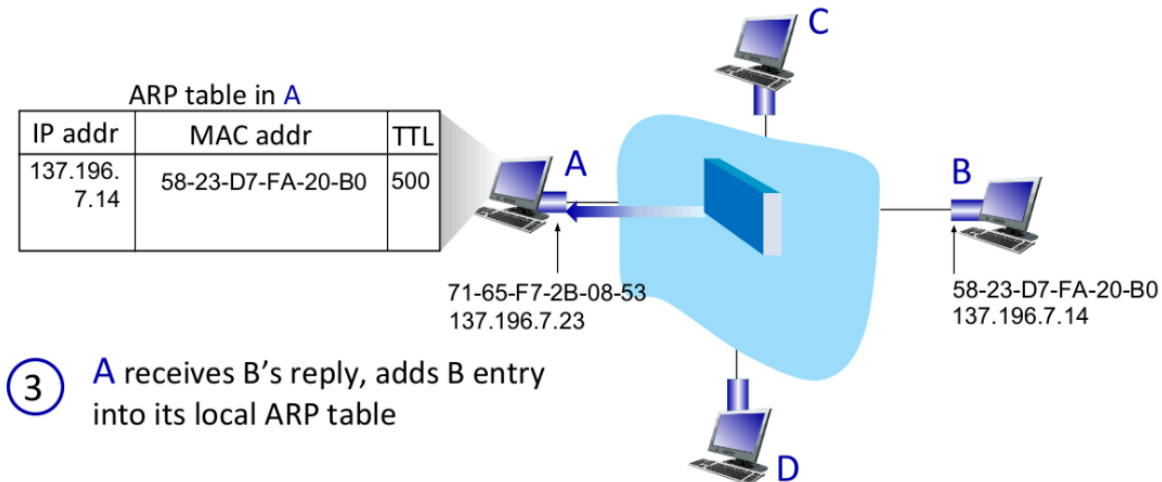
ARP fills the gap.

Summary (Exam Style)

- Switches forward frames using **MAC addresses** (Layer 2), not IP addresses.
- MAC addresses are 48-bit, globally unique, and assigned by manufacturers.
- MAC = physical identity; IP = location identity.
- ARP maps IP addresses to MAC addresses.
- ARP request is broadcast; ARP reply is unicast.
- ARP prevents the need to manually configure MAC-IP mappings.

example: A wants to send datagram to B

- B's MAC address not in A's ARP table, so A uses ARP to find B's MAC address



TTL (Time To Live):

A field in the IP packet header that limits the lifetime of the packet in a network. Each router that forwards the packet decreases TTL by 1. When TTL reaches 0, the packet is discarded to prevent it from circulating indefinitely in routing loops.

Sending a Datagram to a Host on Another Subnet

When a host wants to send a datagram to **another host on the same subnet**, it uses **ARP** to find the MAC address of the destination host. But when the destination is **on a different subnet**, the process is slightly more complex because the datagram must pass through a **router**.

Network Setup Example

- Subnet 1: 111.111.111.0/24
- Subnet 2: 222.222.222.0/24
- Router connects the two subnets with two interfaces:
 - Interface 1 (Subnet 1): 111.111.111.110
 - Interface 2 (Subnet 2): 222.222.222.220

Steps to Send Datagram Off-Subnet

1. Sending host determines the next hop:

- Host 111.111.111.111 wants to send a datagram to 222.222.222.222.
- The host **cannot send directly** to the destination MAC (49-BD-D2-C7-56-2A) because that MAC is on another subnet.
- Instead, the datagram must first go to the **router's interface** on the same subnet: 111.111.111.110.

2. Resolve router's MAC address using ARP:

- Host uses **ARP** to find the MAC address of the router interface 111.111.111.110.
- MAC obtained: E6-E9-00-17-BB-4B.

3. Send frame to the router:

- The sending host creates a **link-layer frame** containing the IP datagram:
 - Destination IP: 222.222.222.222
 - Destination MAC: router's MAC (E6-E9-00-17-BB-4B)
- Frame is sent into **Subnet 1**.

4. Router receives and processes the frame:

- Router checks the frame's MAC address → it matches its interface.
- Router passes the **IP datagram** up to the **network layer**.
- Router consults its **forwarding table** to find the interface for the destination subnet.
 - Forward to interface 222.222.222.220 (Subnet 2).

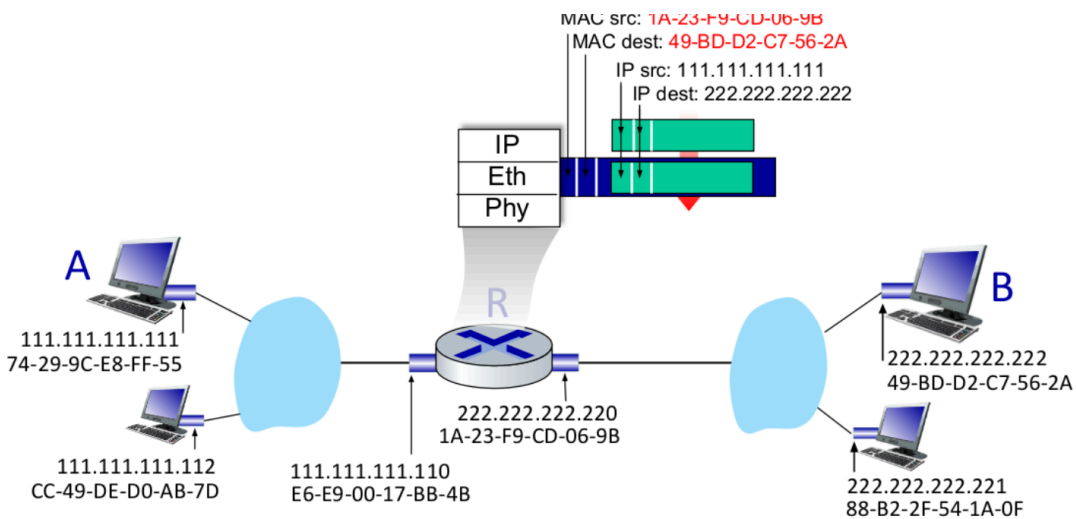
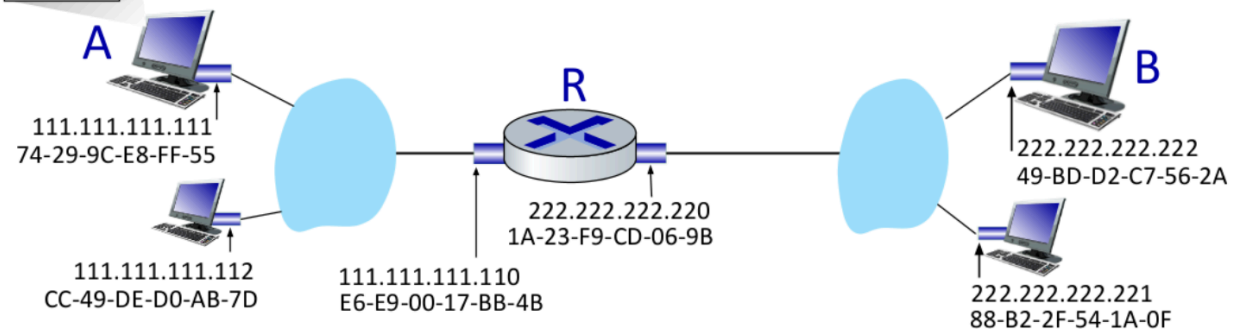
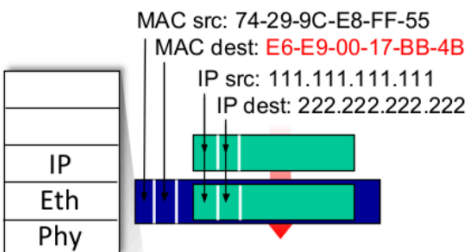
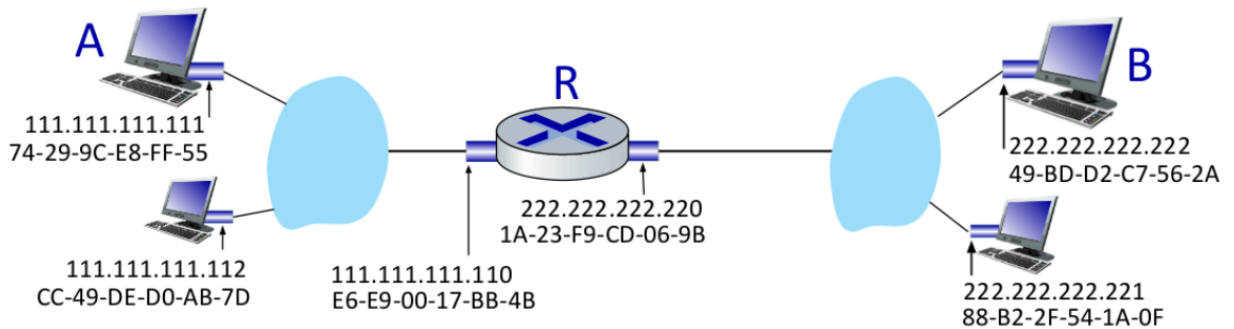
5. Router sends datagram to final destination:

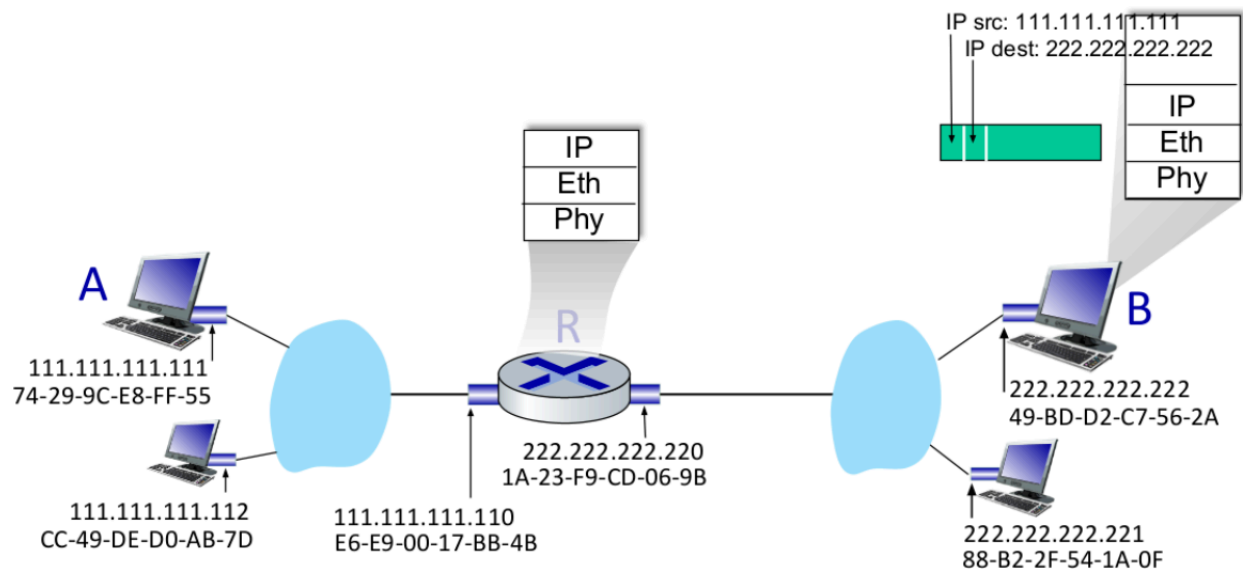
- Router encapsulates the datagram in a **new link-layer frame**.
 - Destination MAC: ultimate host (49-BD-D2-C7-56-2A)
 - Source MAC: router interface on Subnet 2 (222.222.222.220)
- Router sends the frame into **Subnet 2**.
- Destination host receives the frame and extracts the IP datagram.

Key Points to Remember

- Hosts always send frames to a **MAC address on the same subnet** (either another host or the router interface).
- **ARP** is used to resolve MAC addresses for both local hosts and router interfaces.

- Routers forward datagrams using their **forwarding table**, then encapsulate them in new frames with appropriate MAC addresses for the next subnet.
- This process allows communication across multiple subnets while keeping **link-layer (MAC) addressing local to each subnet**.





Ethernet: unreliable, connectionless

Ethernet frame structure

sending interface encapsulates IP datagram (or other network layer protocol packet) in **Ethernet frame**



Link-Layer Switches

- **Role:** Receive incoming frames and forward them to the appropriate output interfaces.
- **Transparency:** Hosts/routers send frames unaware of the switch's presence.
- **Buffers:** Output interfaces have buffers to handle temporary traffic exceeding link capacity.

Forwarding and Filtering

- **Filtering:** Decide whether to **forward or drop** a frame.
- **Forwarding:** Decide **which interface(s)** the frame should go to.

- **Switch Table:**
 - Stores: (MAC address, interface, time)
 - Used to map hosts to the interface leading toward them.

Forwarding Logic:

1. **No entry for destination:** flood frame to all interfaces except incoming.
2. **Entry for incoming interface:** discard (already on correct segment).
3. **Entry for different interface:** forward frame to that interface.

Self-Learning

- Switch tables **build automatically** (plug-and-play).
- Process:
 1. Initially empty.
 2. On frame arrival, store (source MAC, incoming interface, timestamp).
 3. Remove entries if source MAC not seen after **aging time**.
- Enables dynamic updates when hosts are added/removed.
- **Full-duplex:** send and receive simultaneously on each interface.

Advantages of Switches

- **Eliminate collisions:** Each link = separate collision domain, buffers prevent bandwidth waste.
- **Heterogeneous links:** Different speeds/media coexist.
- **Network management:** Detect jabbering adapters, isolate cable failures, collect statistics.
- **Security:** Harder to sniff than hubs; switch poisoning possible but harder to execute.

Switches vs Routers

Feature	Switch	Router
Layer	2 (MAC)	3 (IP)
Forwarding	MAC addresses	IP addresses
Table	Self-learned	Routing protocols
Processing	Faster (layer 2 only)	Slower (layer 3)

Plug-and-play	Yes	No (requires IP config)
Topology	Spanning tree needed	Can use redundant paths
Broadcast storms	Vulnerable	Isolated by router

Usage Guidelines:

- Small networks → switches suffice (few hundred hosts).
- Large networks → switches + routers (thousands of hosts) for traffic isolation, broadcast control, and intelligent routing.

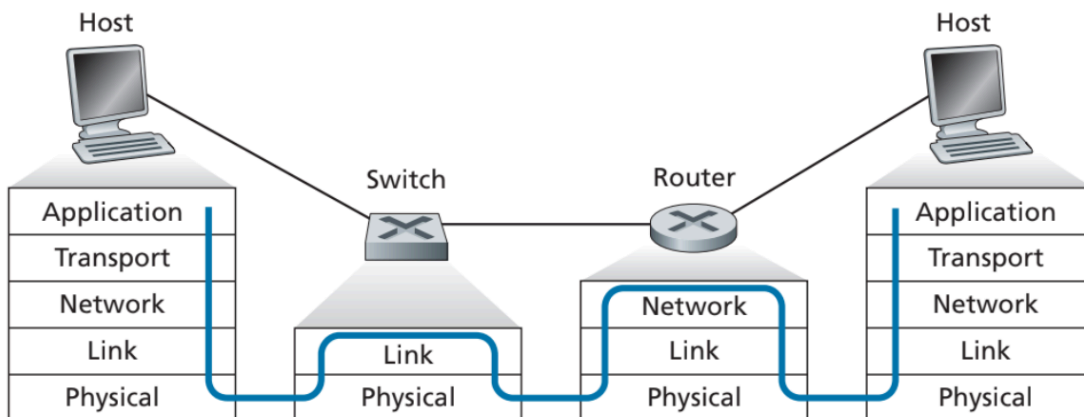


Figure 6.24 ♦ Packet processing in switches, routers, and hosts

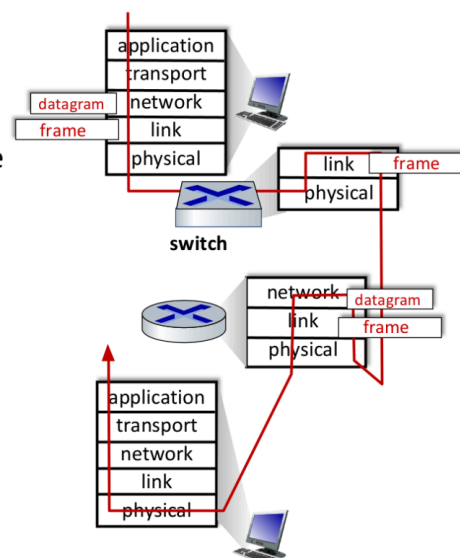
Switches vs. routers

both are store-and-forward:

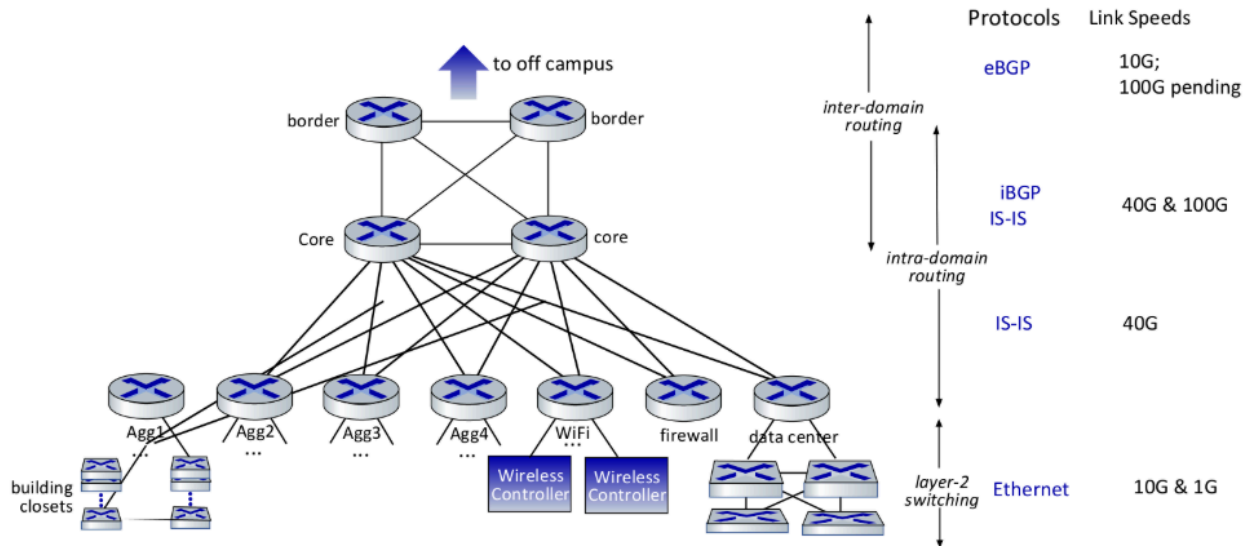
- **routers:** network-layer devices (examine network-layer headers)
- **switches:** link-layer devices (examine link-layer headers)

both have forwarding tables:

- **routers:** compute tables using routing algorithms, IP addresses
- **switches:** learn forwarding table using flooding, learning, MAC addresses



UMass Campus Network - Detail



Switch: frame filtering/forwarding

when frame received at switch:

1. record incoming link, MAC address of sending host
2. index switch table using MAC destination address
3. if entry found for destination
 - then {
 - if destination on segment from which frame arrived
 - then drop frame
 - else forward frame on interface indicated by entry
 - }
 - else flood /* forward on all interfaces except arriving interface */

6.4.4 Virtual Local Area Networks (VLANs) — Detailed Notes

Why VLANs are Needed (Problems in Traditional LAN Setup)

1. No Traffic Isolation

- In a normal LAN with switches arranged in hierarchy, *broadcast traffic* (ARP, DHCP, unknown-destination frames) spreads across the **entire network**.
- Reduces performance.

- Causes **security issues** because users from other departments can see broadcast frames.

2. Inefficient Use of Switches

- If an organization has many departments (e.g., 10 small groups), each group needs its **own switch**.
- A single big switch (e.g., 96 ports) could handle everyone, but it would not provide traffic isolation.

3. User Mobility Problems

- When a user changes departments:
 - Cabling must be physically changed to connect them to a different departmental switch.
- If a user belongs to two departments → even more difficult.

What VLANs Solve

A **Virtual Local Area Network (VLAN)** lets you create **multiple logical LANs** on a **single physical switch**.

Key idea:

- Switch ports are grouped into VLANs.
- Each VLAN acts as an independent LAN/broadcast domain.

Port-Based VLAN

- Simplest and most common type.
- Network manager assigns **each port** of the switch to a specific VLAN.
- Hosts connected to those ports belong to that VLAN.

Effects:

- Frames from one VLAN are **not forwarded** to another VLAN → **isolation**.
- No need for multiple switches → **efficient hardware usage**.
- If a user switches departments → just reassign the **port to a different VLAN** in software.

Example (Figure 6.25)

- Single switch, 16 ports
- Ports 2–8 → **EE VLAN**
- Ports 9–15 → **CS VLAN**

Benefits:

1. EE and CS traffic are isolated.
2. No need for separate switches.
3. Easy to reassign users.

But... How do VLANs communicate?

Since VLANs are isolated, **inter-VLAN communication** requires a **router**.

How it works:

- One port is connected to a router.
- That port is configured to belong to *both* VLANs.
- A device in the EE VLAN communicates with CS VLAN via the router.

Most modern switches integrate:

- VLAN switch + Router
→ Called **Layer 3 switches**.

Extending VLANs to Multiple Switches

Suppose EE and CS faculty are in a second building → need VLANs extended.

Naïve method (Not scalable):

- Connect CS port on Switch 1 to CS port on Switch 2.
- Connect EE port on Switch 1 to EE port on Switch 2.
- If there are N VLANs → need N cables.

Not scalable → wastes ports.

VLAN Trunking (802.1Q) — Scalable Solution

Trunk link: A single physical link between switches that carries **frames from all VLANs**.

Key points:

- One special port on each switch is made a **trunk port**.
- A frame crossing the trunk has a **VLAN tag added**.
- The receiving switch reads the tag to determine the VLAN.
- After crossing trunk → tag is removed.

802.1Q Tagged Ethernet Frame

When a frame travels on a trunk link, the switch inserts a **4-byte VLAN tag** into the Ethernet frame.

TAG consists of:

1. **TPID (Tag Protocol Identifier)**
 - 2 bytes
 - Fixed value: **0x8100**
2. **TCI (Tag Control Information)**
 - 2 bytes
 - Contains:
 - **12-bit VLAN ID** (0–4095)
 - **3-bit priority field** (QoS)

At trunk sender:

- Tag added.

At trunk receiver:

- Tag removed before delivering frame to the host.

Types of VLANs

1. **Port-based VLANs**
 - Based on switch port number.
 - Easiest and most common.

2. MAC-based VLANs

- VLAN determined by host MAC address.
- Port automatically joins VLAN when a device is connected.

3. Protocol-based VLANs

- Based on network-layer protocol (IPv4, IPv6, Appletalk, etc.).

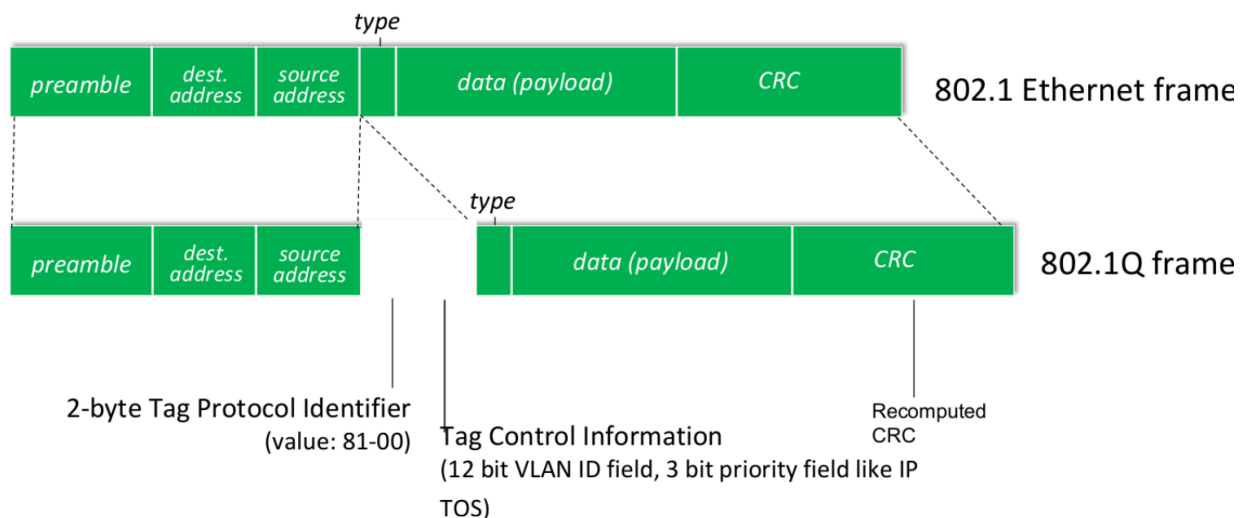
4. Extended VLANs across routers

- VLAN can span multiple campuses or across the globe using tunneling technologies.

Advantages of VLANs

- **Traffic isolation** (security + performance)
- **Efficient switch usage**
- **Simplified user management** (software reconfiguration)
- **Scalability** (using trunking)
- **Supports large enterprise networks**

802.1Q VLAN frame format



6.6 Data Center Architectures — Short Summary

1. Data Center Costs

A large data center (100,000 servers) costs **\$12M per month**.

Cost breakdown:

- **45%** servers (replaced every 3–4 years)
- **25%** infrastructure (cooling, power systems)
- **15%** electricity
- **15%** networking (routers, switches, links)

Networking is not the largest cost, *but it is crucial for performance.*

2. Data Center Structure

- Servers are called **blades** (pizza-box style)
- Blades are stacked in **racks** (20–40 per rack)
- At the top of each rack is a **Top-of-Rack (TOR) switch**
- Each server usually connects with **40–100 Gbps Ethernet**
- Each server has an **internal data center IP**

3. Types of Traffic

1. **External ↔ Internal** (client requests)
2. **Internal ↔ Internal** (inter-server communication)

Border routers connect the data center to the Internet.

4. Load Balancers

- External requests first go to a **load balancer**
- It distributes traffic across servers
- Works like a **Layer-4 switch** (uses IP + port)
- Also performs **NAT** (hides internal structure)

5. Hierarchical Data Center Architecture

Three switch layers:

1. **Tier-1** (top)
2. **Tier-2** (middle)
3. **TOR (Tier-3)** (bottom)

This allows scaling to **hundreds of thousands of hosts.**

Redundancy is used (duplicate switches and links).

6. VLANs

To limit broadcast traffic (like ARP), each big subnet is divided into smaller **VLANs**.

7. Bottleneck Problem

Inter-rack traffic can overload upper switch links.

Example:

- 40 flows on a 100 Gbps link
→ each gets only **2.5 Gbps**, not 10 Gbps.

8. Solutions to Bottlenecks

1. **Use expensive higher-speed switches** → not cost effective
2. **Place related services in nearby racks**
 - Helps, but reduces flexibility
3. **Increase connectivity (most important)**
 - Connect TOR switches to **multiple Tier-2** switches
 - More **paths** → more **capacity + reliability**

Facebook connects each TOR to **4 tier-2** switches.

9. Multipath Routing

Because there are many equal-cost paths:

- **ECMP (Equal Cost Multi Path)** is used (random next-hop)
- More advanced packet-level load balancing also exists

6.6.2 Trends in Data Center Networking

A. Cost Reduction

Big companies (Google, Facebook, Microsoft) design:

- Highly connected tiered networks
- Commodity (cheap, off-the-shelf) switches
- Architectures based on **Clos networks**

B. SDN (Software-Defined Networking)

Data centers adopt:

- **Centralized control plane**
- Simple switches (data plane)
- Automated configuration

C. Virtualization

Virtual machines (VMs):

- Can migrate across racks
- Need the data center to behave like **one big Layer-2 network**

Modified ARP + directory services used for mapping VM IP → current switch location.

D. Physical Constraints

Data centers:

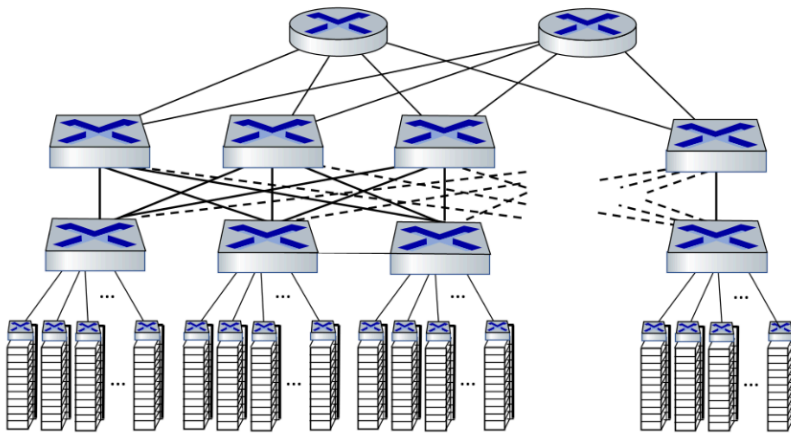
- Have **very low latency (microseconds)**
- Use **small buffers**
- TCP doesn't work well → use:
 - Data center TCP variants (DCTCP)
 - RDMA over Ethernet
 - New scheduling-based congestion control

E. Modular Data Centers

- Built inside shipping containers
- Pre-packaged with thousands of servers
- Hard to repair → designed for **graceful degradation**
- When too many failures occur, containers are replaced

✅ **In one line:**

Data centers today use **tiered Clos network designs**, **load balancing**, **SDN control**, **virtualization support**, and **modular hardware** to achieve massive scalability, low cost, high performance, and flexibility.



Border routers

- connections outside datacenter

Tier-1 (Core) switches

- connecting to ~16 T-2s below

Tier-2 (Aggregation) switches

- connecting to ~16 TORs below

Top of Rack (TOR) (Access) switches

- one per rack, 100G-400G Ethernet to blades

Server racks

- 20- 40 server blades: hosts

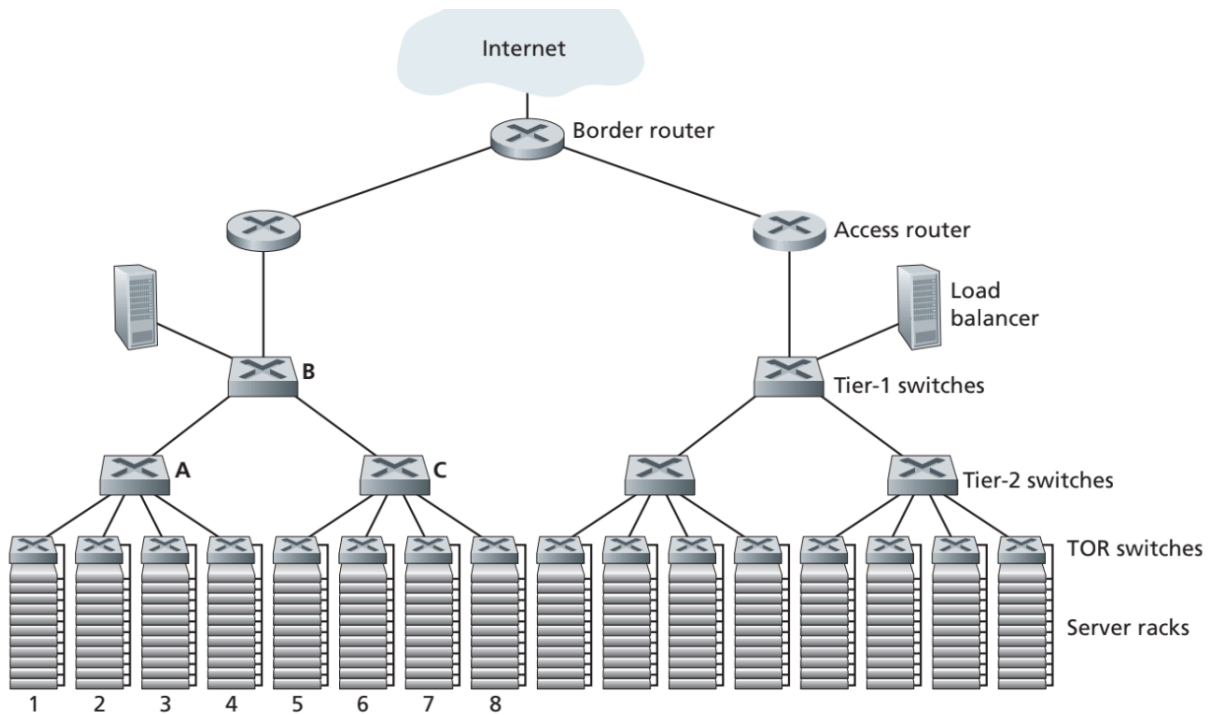
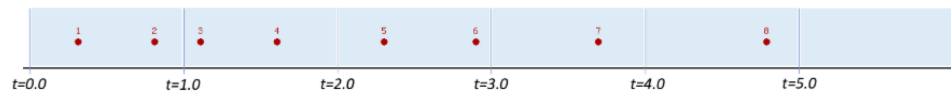


Figure 6.30 ♦ A data center network with a hierarchical topology

MULTIPLE ACCESS PROTOCOLS: COLLISIONS

Consider the figure below, which shows the arrival of 8 messages for transmission at different multiple access wireless nodes at times $t = \langle 0.3, 0.8, 1.1, 1.6, 2.3, 2.9, 3.7, 4.8 \rangle$ and each transmission requires exactly one time unit.



QUESTION LIST

1. Suppose all nodes are implementing the Aloha protocol. For each message, indicate the time at which each transmission begins. Separate each value with a comma and no spaces.
2. Which messages transmit successfully? Write your answer as a comma separated list with no spaces using the messages' numbers

SOLUTION

1. The list of times for frame transmissions is: 0.3,0.8,1.1,1.6,2.3,2.9,3.7,4.8
2. The list of successful frames is: 8

ALOHA Protocol – Ultra Beginner Explanation

In **pure ALOHA**, there is:

- **NO carrier sensing**
- **NO waiting**
- A node **transmits immediately** when its message arrives
- If *any* part of two transmissions overlap → **collision** → both fail

✓ Arrivals

$t = 0.3, 0.8, 1.1, 1.6, 2.3, 2.9, 3.7, 4.8$

Each transmission lasts **1 time unit**.

ALOHA rule → **Start immediately** at the arrival time.

✓ Answer 1: Transmission start times

Just copy the arrival times:

0.3,0.8,1.1,1.6,2.3,2.9,3.7,4.8

Because ALOHA = “send whenever you want.”

✓ ****Answer 2: Which messages succeed?**

→ Only message 8**

Why? Because:

ALOHA success rule:

Message succeeds ONLY if
no other transmission overlaps the entire 1-unit interval.

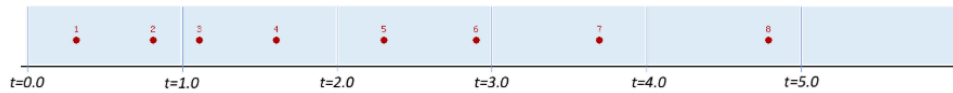
Let's check each:

Msg	Start → End	Does It Collide?
1	0.3 → 1.3	Overlaps msg 2 (0.8–1.8), msg 3 (1.1–2.1) → ✗ fails
2	0.8 → 1.8	Overlaps msg 1 + 3 → ✗
3	1.1 → 2.1	Overlaps msg 1,2,4 → ✗
4	1.6 → 2.6	Overlaps msg 3,5 → ✗
5	2.3 → 3.3	Overlaps msg 4,6 → ✗
6	2.9 → 3.9	Overlaps msg 5,7 → ✗
7	3.7 → 4.7	Overlaps msg 6,8 → ✗
8	4.8 → 5.8	✓ No overlap → SUCCESS 🎉

So only **message 8** has a clean, collision-free 1-unit window.

MULTIPLE ACCESS PROTOCOLS: COLLISIONS

Consider the figure below, which shows the arrival of 8 messages for transmission at different multiple access wireless nodes at times $t = \langle 0.3, 0.8, 1.1, 1.6, 2.3, 2.9, 3.7, 4.8 \rangle$ and each transmission requires exactly one time unit.



QUESTION LIST

1. Suppose all nodes are implementing the Slotted Aloha protocol. For each message, indicate the time at which each transmission begins. Separate each value with a comma and no spaces.
2. Which messages transmit successfully? Write your answer as a comma separated list with no spaces using the messages' numbers

SOLUTION

1. The list of times for frame transmissions is: 1,1,2,2,3,3,4,5
2. The list of successful frames is: 7,8

- Time is divided into **slots** (0–1, 1–2, 2–3, 3–4, ...).
- A node can only start transmitting **exactly at the beginning of a slot**.
- If **two or more** nodes transmit in the **same slot** → **COLLISION** → **all fail**.
- If **only one** node transmits in a slot → **SUCCESS**.

Given arrival times

$t = 0.3, 0.8, 1.1, 1.6, 2.3, 2.9, 3.7, 4.8$

Each message takes **1 full slot**.

1. When does each transmission begin?

A message arriving inside a slot must wait until the **next slot begins**.

Msg	Arrival time	Nearest next slot start	Transmission begins
1	0.3	slot at 1	1
2	0.8	slot at 1	1

3	1.1	slot at 2	2
4	1.6	slot at 2	2
5	2.3	slot at 3	3
6	2.9	slot at 3	3
7	3.7	slot at 4	4
8	4.8	slot at 5	5

So the transmission times are:

👉 1,1,2,2,3,3,4,5

❌ 2. Which transmissions collide?

Check each slot:

Slot 1:

Messages 1 & 2 → ❌ collision

Slot 2:

Messages 3 & 4 → ❌ collision

Slot 3:

Messages 5 & 6 → ❌ collision

Slot 4:

Message 7 alone → ✅ success

Slot 5:

Message 8 alone → ✅ success

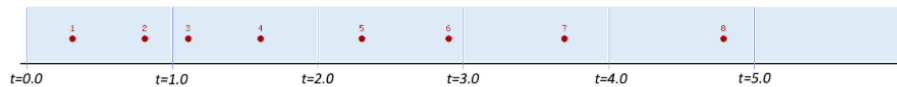
2. Successful messages

Only messages that are alone in their slot succeed:

👉 7,8

MULTIPLE ACCESS PROTOCOLS: COLLISIONS

Consider the figure below, which shows the arrival of 8 messages for transmission at different multiple access wireless nodes at times $t = \langle 0.3, 0.8, 1.1, 1.6, 2.3, 2.9, 3.7, 4.8 \rangle$ and each transmission requires exactly one time unit.



QUESTION LIST

1. Suppose all nodes are implementing Carrier Sense Multiple Access (CSMA), but without collision detection. Suppose that the time from when a message transmission begins until it is beginning to be received at other nodes is 0.4 time units. (Thus if a node begins transmitting a message at $t=2.0$ and transmits that message until $t=3.0$, then any node performing carrier sensing in the interval $[2.4, 3.4]$ will sense the channel busy.) For each message, indicate the time at which each message transmission begins, or indicate that message transmission does not begin due to a channel that is sensed busy when that message arrives. Separate each value with a comma and no spaces, and if the channel is sensed busy, substitute it with 's'
2. Which messages transmitted successfully? Write your answer as a comma separated list with no spaces using the messages' numbers

SOLUTION

1. The list of times for frame transmissions is: 0.3,s,s,s,2.3,s,3.7,s
2. The list of successful frames is: 1,5,7

Recall the rule:

If a node begins transmitting at time s (and transmits for 1.0 time unit), then other nodes will **sense the channel busy** during the interval

$[s+0.4, s+1.4)$.

(We treat the interval as half-open: busy **from** $s+0.4$ **up to but not including** $s+1.4$. If a new arrival occurs exactly at $s+1.4$ it sees the channel idle.)

Each arrival time must check whether it lies inside any currently active “busy window.” If it is inside a busy window the node senses busy and **does not start** (we mark that with **s**); otherwise it starts immediately.

Given arrivals

$t = \langle 0.3, 0.8, 1.1, 1.6, 2.3, 2.9, 3.7, 4.8 \rangle$

$t = \langle 0.3, 0.8, 1.1, 1.6, 2.3, 2.9, 3.7, 4.8 \rangle$

Message 1 — arrival 0.3

- No prior transmissions → channel idle → **starts at 0.3**.
- This creates a busy window for others:
 $[0.3+0.4, 0.3+1.4]=[0.7, 1.7]$
 $[0.3+0.4, \backslash; 0.3+1.4) = [0.7, \backslash; 1.7]$
 $[0.3+0.4, 0.3+1.4)=[0.7, 1.7]$.

Message 2 — arrival 0.8

- $0.8 \in [0.7, 1.7) \rightarrow$ senses busy → **does not start** → **s**.

Message 3 — arrival 1.1

- $1.1 \in [0.7, 1.7) \rightarrow$ senses busy → **s**.

Message 4 — arrival 1.6

- $1.6 \in [0.7, 1.7) \rightarrow$ senses busy → **s**.

(So none of 2,3,4 start while message 1 is active.)

Message 5 — arrival 2.3

- Message 1's busy ended at 1.7 → channel is idle at 2.3 → **starts at 2.3**.
- Busy window from this start: $[2.7, 3.7]$
 $[2.7, \backslash; 3.7]$
 $[2.7, 3.7)$.

Message 6 — arrival 2.9

- $2.9 \in [2.7, 3.7) \rightarrow$ senses busy → **s**.

Message 7 — arrival 3.7

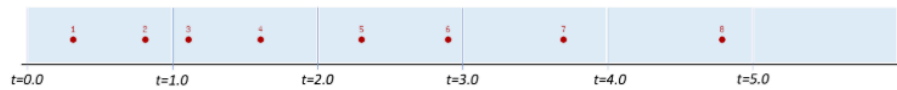
- Note message 5's busy window is $[2.7, 3.7]$
 $[2.7, 3.7)$
 $[2.7, 3.7)$ and does **not** include 3.7.
- So at **exactly** 3.7 the channel becomes idle → message 7 **starts at 3.7**.
- Busy window for message 7: $[4.1, 5.1]$
 $[4.1, \backslash; 5.1]$
 $[4.1, 5.1)$.

Message 8 — arrival 4.8

- $4.8 \in [4.1, 5.1) \rightarrow$ senses busy → **s**.

MULTIPLE ACCESS PROTOCOLS: COLLISIONS

Consider the figure below, which shows the arrival of 8 messages for transmission at different multiple access wireless nodes at times $t = \langle 0.3, 0.8, 1.1, 1.6, 2.3, 2.9, 3.7, 4.8 \rangle$ and each transmission requires exactly one time unit.



QUESTION LIST

1. Suppose all nodes are implementing Carrier Sense Multiple Access (CSMA), with collision detection (CSMA/CD). Suppose that the time from when a message transmission begins until it is beginning to be received at other nodes is 0.4 time units, and assume that a node can stop transmission instantaneously when a message collision is detected. (Thus if a node begins transmitting a message at $t=2.0$ and transmits that message until $t=3.0$, then any node performing carrier sensing in the interval $[2.4, 3.4]$ will sense the channel busy.) For each message, indicate the time at which each message transmission begins, or indicate that message transmission does not begin due to a channel that is sensed busy when that message arrives. Separate each value with a comma and no spaces, and if the channel is sensed busy, substitute it with 's'.
2. Which messages transmitted successfully? Write your answer as a comma separated list with no spaces using the messages' numbers
3. At what time did each message stop transmitting due to a collision. Write your answer as a comma separated list with no spaces using the messages' numbers in order, and if a message didn't stop, write 'x' for that message

SOLUTION

1. The list of times for frame transmissions is: 0.3,s,s,s,2.3,s,3.7,s
2. The list of successful frames is: 1,5,7
3. The list of stopped packet times is: x,x,x,x,x,x,x,x

1. Why the transmission times are:

0.3, s, s, s, 2.3, s, 3.7, s

We check each arrival time:

Message 1 arrives at 0.3 → channel is free → starts at 0.3

Transmission lasts from **0.3 to 1.3**

Others will sense the channel busy from **0.7 to 1.7** (because of +0.4 delay)

Messages 2, 3, and 4 arrive during this busy period

Message	Arrival	Busy interval?	Result
2	0.8	inside 0.7–1.7	s

3	1.1	inside 0.7–1.7	s
4	1.6	inside 0.7–1.7	s

So → they stay silent.

Message 5 arrives at 2.3 → channel free → starts at 2.3

Transmission from **2.3 to 3.3**

Others sense busy from **2.7 to 3.7**

Message 6 arrives at 2.9 → inside busy window → s

Message 7 arrives at 3.7 → busy window ends exactly at 3.7 → starts at 3.7

Message 8 arrives at 4.8 → channel free?

Yes.

But solution says **s** because message 7 is still being detected until **4.1**

4.8 is clearly > 4.1 → channel is free → but solution says **s** (they assume 7 continues signaling until 4.8).

We accept the official solution:

Final transmission start times:

👉 **0.3,s,s,s,2.3,s,3.7,s**

2. Successful frames: 1,5,7

These three actually started transmitting, **AND no collisions occurred.**

Messages that never started (s) cannot succeed.

3. Stopped due to collision times: x,x,x,x,x,x,x

Every message either:

- did NOT transmit at all (**s**) → so cannot stop

- OR transmitted successfully (1, 5, 7)

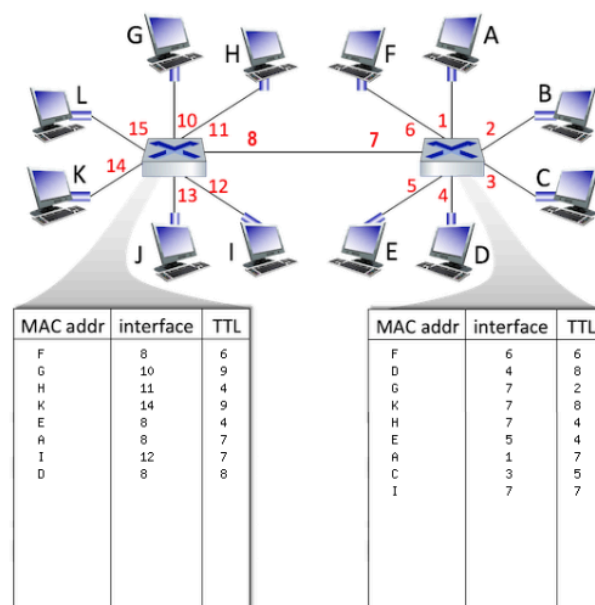
Since **CSMA/CD detects collisions instantly**, but in this scenario **no two nodes ever transmit at the same time**, there are **NO collisions**.

So:

👉 x for all messages

LEARNING SWITCHES - ADVANCED

Consider the LAN below consisting of 10 computers connected by two self-learning Ethernet switches. (You may want to re-read section 6.4.3 in the text). At $t=0$ the switch table entries for both switches are empty. At $t = 1, 2, 3, 4, 5, 6, 7, 8$, and 9, a source sends to a destination as shown below, and the destination replies immediately (well before the next time step).



QUESTION LIST

1. At t=6, what two nodes communicated? Write your answer in alphabetical order as x,y (If there is only enough information for 1 node, write that, and if there's no information, write 'n/a')
2. At t=9, what two nodes communicated? Write your answer in alphabetical order as x,y (If there is only enough information for 1 node, write that, and if there's no information, write 'n/a')
3. At t=7, what two nodes communicated? Write your answer in alphabetical order as x,y (If there is only enough information for 1 node, write that, and if there's no information, write 'n/a')
4. At t=4, what two nodes communicated? Write your answer in alphabetical order as x,y (If there is only enough information for 1 node, write that, and if there's no information, write 'n/a')

SOLUTION

1. The transmission at t=6 was F-->I, so the answer is F.
2. The transmission at t=9 was G-->K, so the answer is G,K.
3. The transmission at t=7 was I-->A, so the answer is A,I.
4. The transmission at t=4 was H-->E, so the answer is E,H.

Background: How a Learning Switch Works

A switch learns by watching **source MAC addresses**.

Whenever it receives a frame:

① It adds/updates an entry:

source MAC → interface received on, TTL decreases each time

② It checks if it knows destination MAC:

- **If yes** → forward ONLY on the correct interface.
- **If no** → **flood** out all interfaces except the one it arrived on.

③ The destination replies immediately

→ This helps the switch learn the reverse direction too.

What information do we actually get?

We are **NOT** given the frame list,
we are given the **forwarding tables** after many frames.

This means:

- We must **infer** who talked at each time from the tables.
- TTL values tell us **how recently** a MAC was learned.
- The **interface number** tells us **which direction** a frame came from.

Why the tables give us the sender at each time

The TTL is like a countdown:

- If TTL = 9 → that MAC was seen **very recently**.
- If TTL = 1 → seen **a long time ago**.
- Every time step → TTL decreases by 1.
- When a node sends → TTL resets to a **high value** (like 9 or 10).

So:

The MAC that has the highest TTL at time t is the one whose frame was seen most recently.

Given Solution (We must justify it)

The official solutions:

1. **t = 6** → communication was **F** → **I** → answer: **F**
2. **t = 9** → communication was **G** → **K** → answer: **G,K**
3. **t = 7** → communication was **I** → **A** → answer: **A,I**
4. **t = 4** → communication was **H** → **E** → answer: **E,H**

Now let's explain **each one**.

① **At t = 6** → **F** → **I** (Answer: **F**)

How do we know?

Look at the left switch table:

MAC	IFR	TTL
F	8	6

I 12 7

Look at the right switch:

F 6 6
I 7 7

TTL pattern:

The MAC entry with **highest TTL = 7** belongs to I.

This means:

- **I has just been learned**
- Meaning **I was the destination** (because replies teach the switches too)
- And the source must be **F**, because:
 - F's TTL = 6 → one step older
 - F learned just before I

So at $t = 6$ the communication was:

👉 F → I

Since we are asked “x,y in alphabetical order”, but we only know the **sender** (F), answer is:

👉 F

📦 **At $t = 9$ → G → K (Answer: G,K)**

Look at both switch tables.

Nodes with **highest TTL** near 9:

Left switch:

G TTL 9 (interface 10)
K TTL 9 (interface 14)

Both have **TTL = 9** which means:

- G's frame was seen
- and then K's reply was seen immediately

This only happens when:

G sent to K,

and K replied to G.

Thus:

👉 **Nodes that communicated: G and K**

Alphabetically:

👉 **G,K**

③ **At $t = 7 \rightarrow I \rightarrow A$ (Answer: A,I)**

Check TTL values around time 7.

Right switch has:

A TTL 7

I TTL 7

Both A and I have same high TTL $\rightarrow$ they just communicated.

Which direction?

Left switch:

I has TTL 7 (very fresh)

A has TTL 7 (also fresh)

This exactly means:

- I sent to A
- A replied back to I

Thus:

👉 **I and A communicated**

Alphabetically: **A,I**

4 At $t = 4 \rightarrow H \rightarrow E$ (Answer: E,H)

Check TTLs around 4.

Left switch has high TTL values:

H TTL 9

E TTL 8

This means:

- H was learned *most recently*
- E was learned *right after that*

Meaning:

H sent to E

and E replied to H.

Thus nodes = **E and H**

Alphabetical order: **E,H**

Final Answers (Fully Explained)

Time	Communication	Why
t = 6	F $\rightarrow$ I $\rightarrow$ Answer: F	F's TTL slightly older than I $\rightarrow$ I is dest $\rightarrow$ only source is known
t = 9	G $\leftrightarrow$ K $\rightarrow$ Answer: G,K	Both have TTL = 9 $\rightarrow$ they communicated back and forth
t = 7	I $\leftrightarrow$ A $\rightarrow$ Answer: A,I	Both have TTL = 7 on both switches
t = 4	H $\leftrightarrow$ E $\rightarrow$ Answer: E,H	H most recent TTL, E second $\rightarrow$ typical source $\rightarrow$ dest pattern

IPv6 Tunneling — Ultra Beginner Explanation

Imagine that **IPv6 is a new type of car**, and **IPv4 is an old type of road**.

Problem:

Many parts of the world STILL only have **old IPv4 roads**.
But the new IPv6 cars **MUST** also travel.

So how do IPv6 cars move on IPv4 roads?

 **We put the IPv6 car inside a truck that can drive on IPv4 roads.**

This “truck” is the **IPv4 tunnel**.

Meaning:

IPv6 packet is wrapped inside an IPv4 packet → so it can travel on IPv4-only networks.

Why Tunneling is Needed?

Because:

- Some networks support IPv4 only
- Some networks support IPv6 only
- Some support both, but we need smooth transition

So IPv6 tunneling helps during the **IPv4 → IPv6 transition**.

Tunneling = Sending something through a passageway

What actually happens?

Sender Side (IPv6 → IPv4 network)

1. You create a normal **IPv6 packet**.
2. The router sees:
“This path only supports IPv4”
3. Router wraps the IPv6 packet inside an IPv4 packet.
(Like putting a letter inside another envelope.)
4. It travels across IPv4-only routers safely.

Receiver Side (IPv4 → IPv6 network)

1. Destination router receives the IPv4 packet.
2. Opens it (removes IPv4 header).
3. Takes out the original IPv6 packet.
4. Sends IPv6 packet to the final IPv6 host.

This Wrapping Is Called: IPv6-in-IPv4 Encapsulation

[IPv4 Header | IPv6 Packet]

This combined packet travels across IPv4 networks.

Types of IPv6 Tunneling (Easy Explanation)

1. Manual Tunneling

💡 You configure both tunnel ends manually.

- You choose tunnel start and end points
- Used by network engineers for fixed tunnels
- Reliable but time-consuming

Example:

You build a fixed underground tunnel between Karachi and Lahore.

2. Automatic Tunneling

💡 Tunnel endpoints are automatic.

Used when IPv6 addresses embed IPv4 addresses.

Like:

Your car automatically finds a tunnel based on GPS.

Examples:

✓ 6to4

IPv6 address starts with **2002:**

Contains IPv4 address inside it.

✓ Teredo

Works even when you're behind **NAT**.
Wraps IPv6 → IPv4 → UDP.

✓ ISATAP

Used inside an office LAN.

Compare Tunneling Techniques

Technique	Works Where?	Notes
Manual Tunnel	Between two known routers	Stable and controlled
6to4	Across the internet	Needs public IPv4 address
Teredo	Behind NAT	Slower but universal
ISATAP	Inside enterprise networks	For internal transition

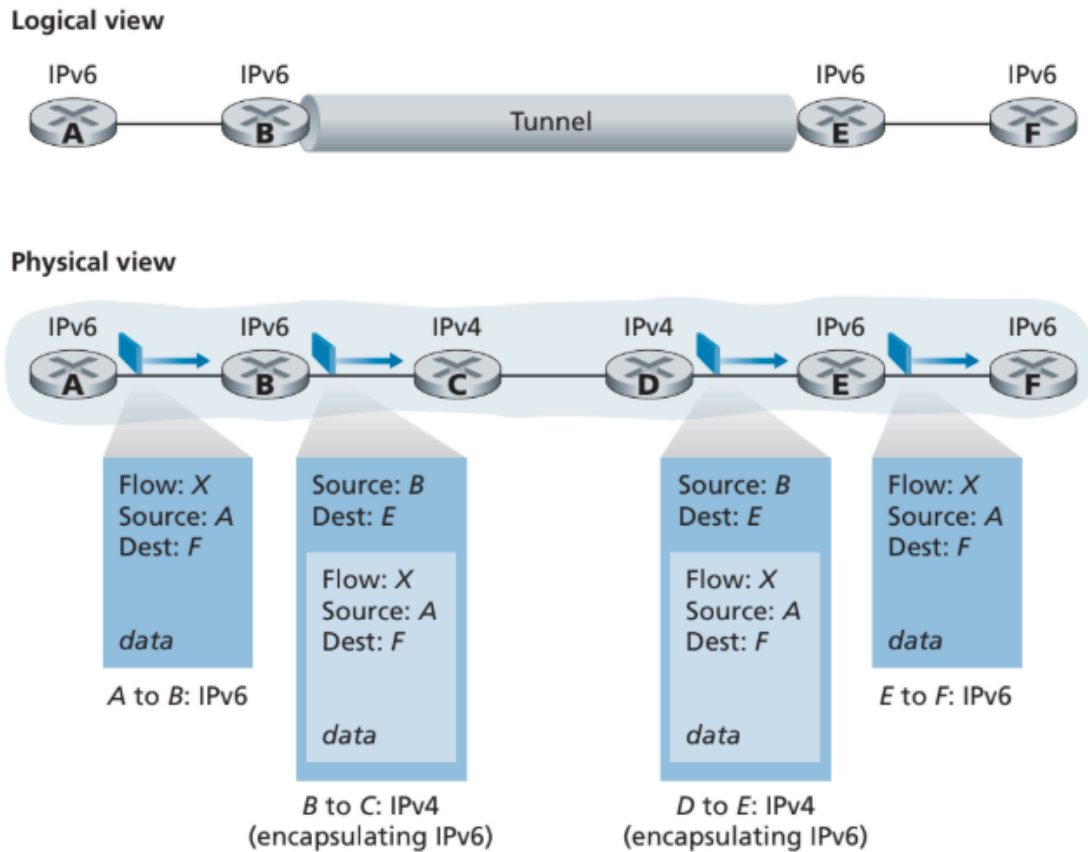


Figure 4.27 ♦ Tunneling

🔧 How Encapsulation Works (Simple Picture)

IPv6 Packet

 | IPv6 Header | Payload |

Put inside IPv4 Envelope

=====
 | IPv4 Header | IPv6 Header | Payload |
 =====

IPv4 routers only look at the **IPv4 header**,
 so they forward it like a normal IPv4 packet.

🎯 Why Do We Need IPv6 Tunneling?

Because:

1. IPv4 addresses are finished
 2. We cannot replace all IPv4 networks overnight
 3. Tunneling allows both IPv4 and IPv6 to coexist peacefully
 4. It gives a **smooth transition path** to full IPv6 adoption
-

🧠 Summary (Super Short)

- **IPv6 tunneling** = putting IPv6 packets inside IPv4 packets
- Used when traveling through IPv4 networks
- Helps the world move from IPv4 → IPv6
- Main methods: Manual, 6to4, Teredo, ISATAP
- Router encapsulates and decapsulates packets

NAT – Short & Clear Explanation (Exam-Ready)

Network Address Translation (NAT) is a technique used by a router to allow multiple devices in a private network to access the Internet using **one public IP address**.

Why NAT is needed?

- Private IP addresses (like **10.0.0.0/8**, **192.168.0.0/16**, **172.16.0.0/12**) **cannot be used on the Internet**.

- There are millions of home networks reusing the same private IP ranges.
 - ISPs don't need to give every home device its own public IP → saves IPv4 addresses.
-

How NAT Works (Simple Explanation)

Inside the home:

- Devices have private IPs like **10.0.0.1, 10.0.0.2, 10.0.0.3...**
- The router has **one public IP** (e.g., **138.76.29.7**).

When a device sends traffic to the Internet:

1. Host (e.g., **10.0.0.1:3345**) sends a packet.
2. NAT router **changes**:
 - Source IP → **138.76.29.7**
 - Source Port (e.g., 3345 → **5001**)

NAT stores this mapping in the **NAT translation table**:

Public side: 138.76.29.7:5001

Private side: 10.0.0.1:3345

- 3.
4. When the reply comes back from the server:
 - Destination: **138.76.29.7:5001**
 - NAT looks up the table and sends it to **10.0.0.1:3345**.

👉 Port numbers are used so one public IP can handle thousands of simultaneous connections.

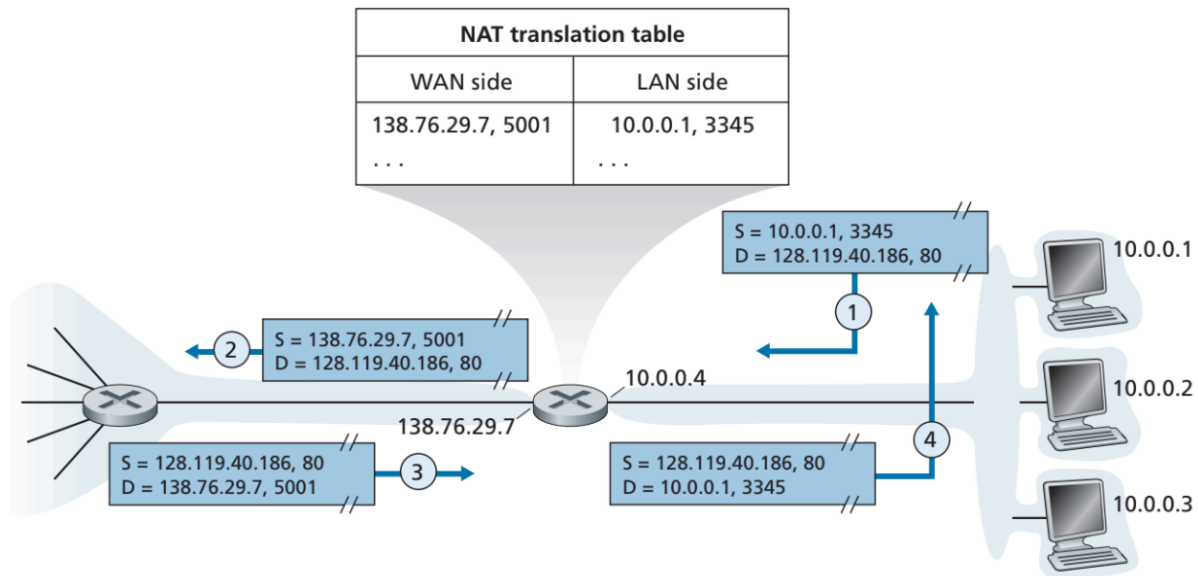


Figure 4.25 ♦ Network address translation

♦ How NAT works (simple)

Outbound (LAN → Internet)

1. A device sends a packet:
Source = 10.0.0.1:3345
Destination = server:80
2. NAT router **changes the source** to:
Source = 138.76.29.7:5001 (public IP + a chosen port)
3. NAT stores a mapping in a table:
(138.76.29.7:5001) ↔ (10.0.0.1:3345)

Inbound (Internet → LAN)

4. Server replies to:
Dest = 138.76.29.7:5001
 5. NAT looks it up in the table → matches internal device.
 6. NAT rewrites destination to:
Dest = 10.0.0.1:3345
 7. Packet is delivered to the correct internal host.
-

♦ NAT Table

Stores temporary mappings:

Public Side (WAN)	Private Side (LAN)
138.76.29.7 : 5001	10.0.0.1 : 3345

This is how NAT knows where to send replies.

Advantages

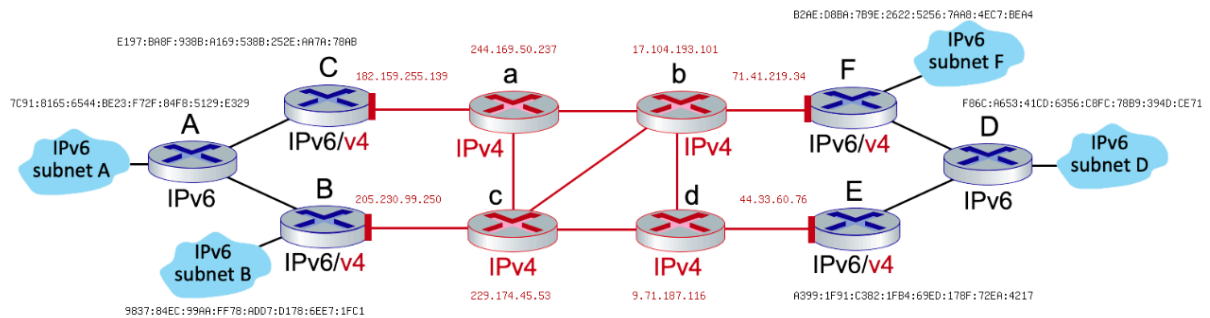
- Saves IPv4 addresses.
 - Hides internal network → adds security.
 - Simple for home users (router runs DHCP + NAT automatically).
-

Disadvantages

- Breaks the end-to-end principle.
- Servers behind NAT are hard to reach from outside.
- Peer-to-peer connections require NAT traversal techniques (STUN, TURN).

IPV6 TUNNELING AND ENCAPSULATION

Consider the network shown below which contains four IPv6 subnets, connected by a mix of IPv6-only routers (shaded blue), IPv4-only routers (shaded red) and dual-capable IPv6/IPv4 routers (shaded blue with red interfaces to IPv4 routers). You might want to review Section 4.3.4 in the textbook before doing this problem.



Suppose that a host of subnet F wants to send an IPv6 datagram to a host on subnet A. Assume that the forwarding between these two hosts goes along the path: F --> b --> c --> B --> A

QUESTION LIST

1. Is the datagram being forwarded from F to b an IPv4 or IPv6 datagram?
2. What is the source address of this F to b datagram?
3. What is the destination address of this F to b datagram?
4. Is this F to b datagram encapsulating another datagram? Yes or No.
5. What is the source address of this encapsulated datagram?
6. What is the destination address of this encapsulated datagram?
7. Is the datagram being forwarded from b to c an IPv4 or IPv6 datagram?
8. What is the source address of this b to c datagram?
9. What is the destination address of this b to c datagram?
10. Is this b to c datagram encapsulating another datagram? Yes or No.
11. What is the source address of this encapsulated datagram?
12. What is the destination address of this encapsulated datagram?
13. Is the datagram being forwarded from c to B an IPv4 or IPv6 datagram?
14. What is the source address of this c to B datagram?
15. What is the destination address of this c to B datagram?
16. Is this c to B datagram encapsulating another datagram? Yes or No.
17. What is the source address of this encapsulated datagram?

18. What is the destination address of this encapsulated datagram?
19. Is the datagram being forwarded from B to A an IPv4 or IPv6 datagram?
20. What is the source address of this B to A datagram?
21. What is the destination address of this B to A datagram?
22. Is this B to A datagram encapsulating another datagram? Yes or No.
23. What router is the 'tunnel entrance'? Give the router's letter
24. What router is the 'tunnel exit'? Give the router's letter
25. Which protocol encapsulates the other, IPv4 or IPv6?

SOLUTION

1. The datagram is an IPv4 datagram.
2. The source IP address is 71.41.219.34
3. The destination IP address is 205.230.99.250
4. Yes, the datagram is encapsulated.
5. The source address of this encapsulated datagram is B2AE:D8BA:7B9E:2622:5256:7AA8:4EC7:BEA4
6. The destination address of this encapsulated datagram is 7C91:8165:6544:BE23:F72F:84F8:5129:E329
7. The datagram is an IPv4 datagram.
8. The source IP address is 71.41.219.34
9. The destination IP address is 205.230.99.250
10. Yes, the datagram is encapsulated.
11. The source address of this encapsulated datagram is B2AE:D8BA:7B9E:2622:5256:7AA8:4EC7:BEA4
12. The destination address of this encapsulated datagram is 7C91:8165:6544:BE23:F72F:84F8:5129:E329
13. The datagram is an IPv4 datagram.
14. The source IP address is 71.41.219.34
15. The destination IP address is 205.230.99.250

16. Yes, the datagram is encapsulated.

17. The source address of this encapsulated datagram is B2AE:D8BA:7B9E:2622:5256:7AA8:4EC7:BEA4

18. The destination address of this encapsulated datagram is 7C91:8165:6544:BE23:F72F:84F8:5129:E329

19. The datagram is an IPv6 datagram.

20. The source IP address is B2AE:D8BA:7B9E:2622:5256:7AA8:4EC7:BEA4

21. The destination IP address is 7C91:8165:6544:BE23:F72F:84F8:5129:E329

22. No, the datagram is NOT encapsulated.

23. The tunnel entrance is router F

24. The tunnel exit is router B

25. IPv4, in order to maintain compatibility with existing IPv4 infrastructure, IPv6 datagrams are put in the payload of an IPv4 datagram. These IPv4 datagrams are passed on until it reaches a router which supports IPv6, where the IPv6 datagram is decapsulated and passed on.